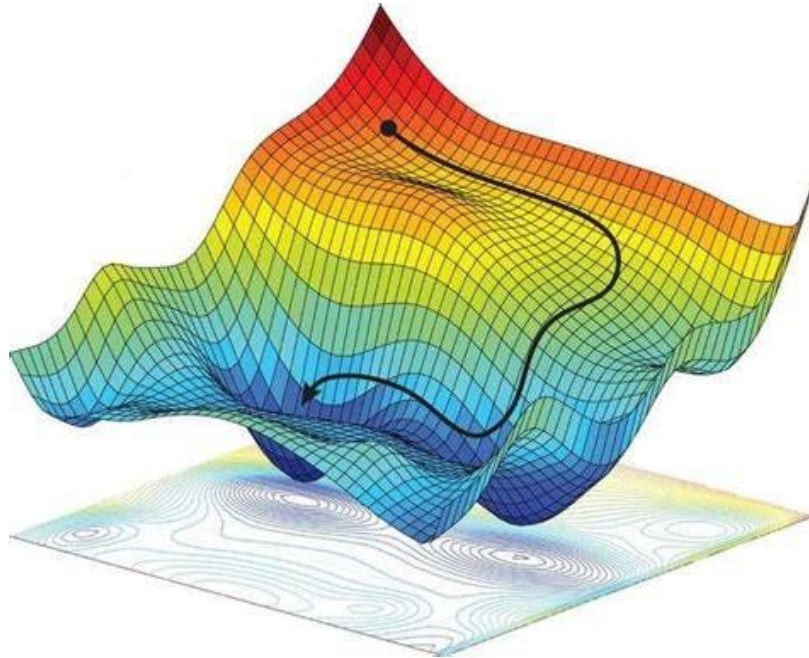


Artificial Intelligence and Machine Learning

Week 06

Practical Aspects of Deep Learning



Before Training

Three way hold out Method

Train Set: Used for updating weights through backpropagation.

Validation Set: Used to monitor the model during training and detect overfitting.

Test Set: Used only after training to evaluate generalization performance.

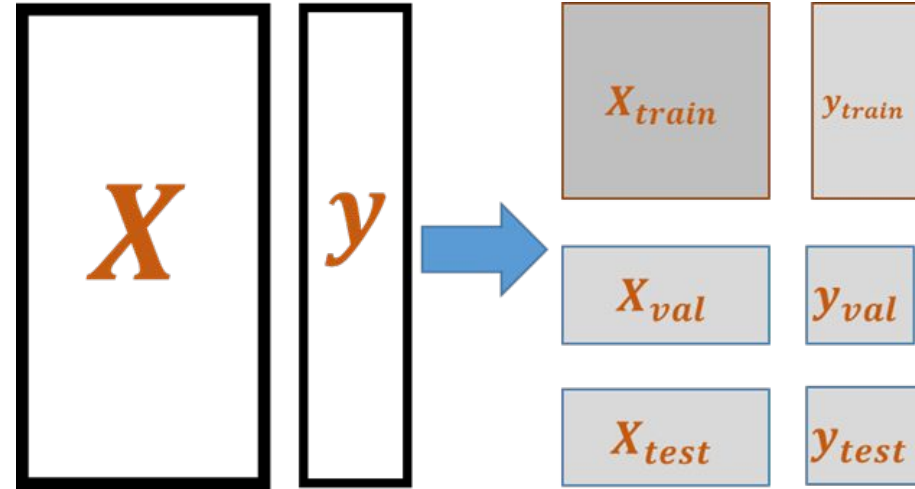
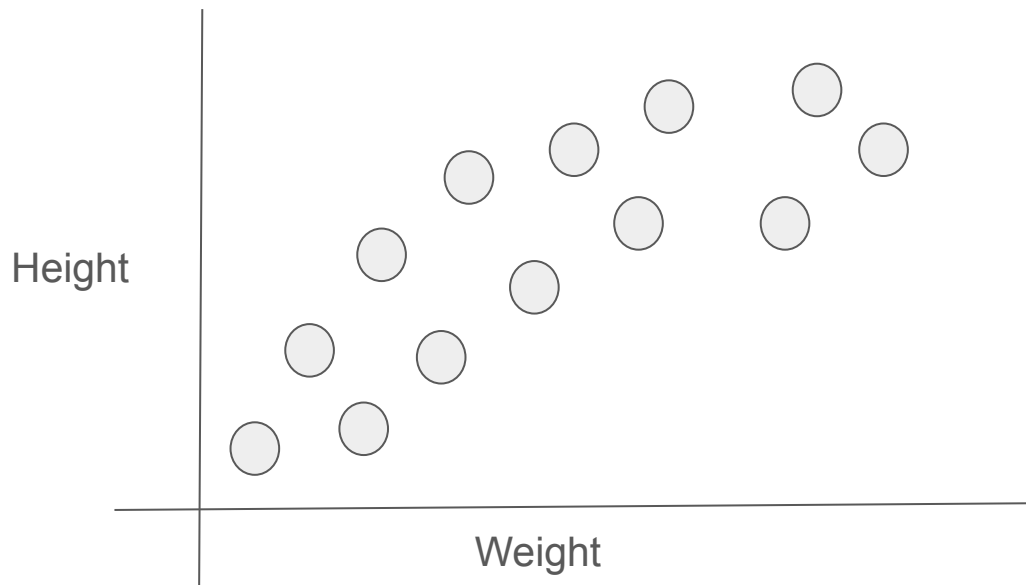


Fig: Three Way Holdout Methods.

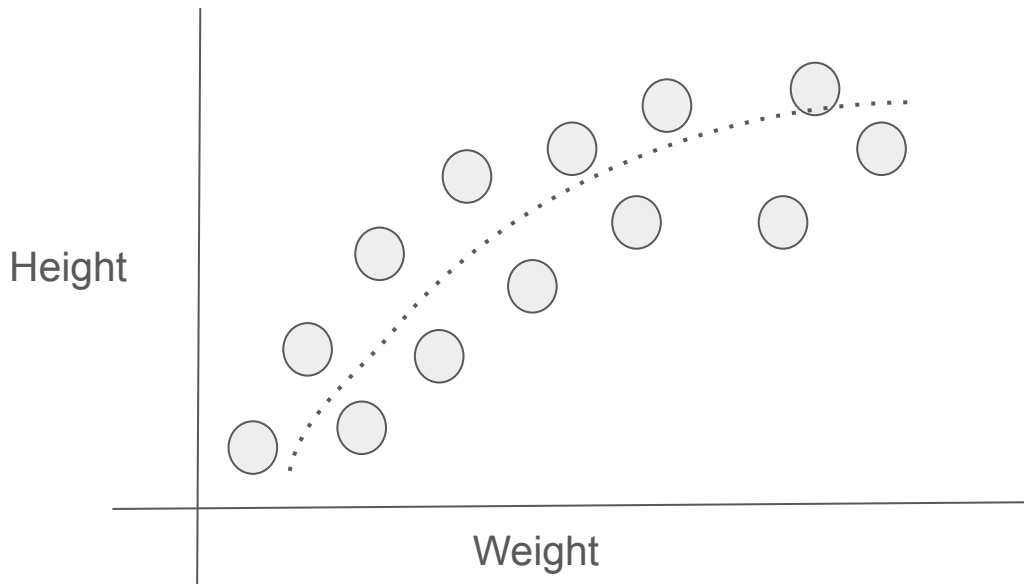
Bias vs Variance

Let's say we have a bunch of weight data and the corresponding heights



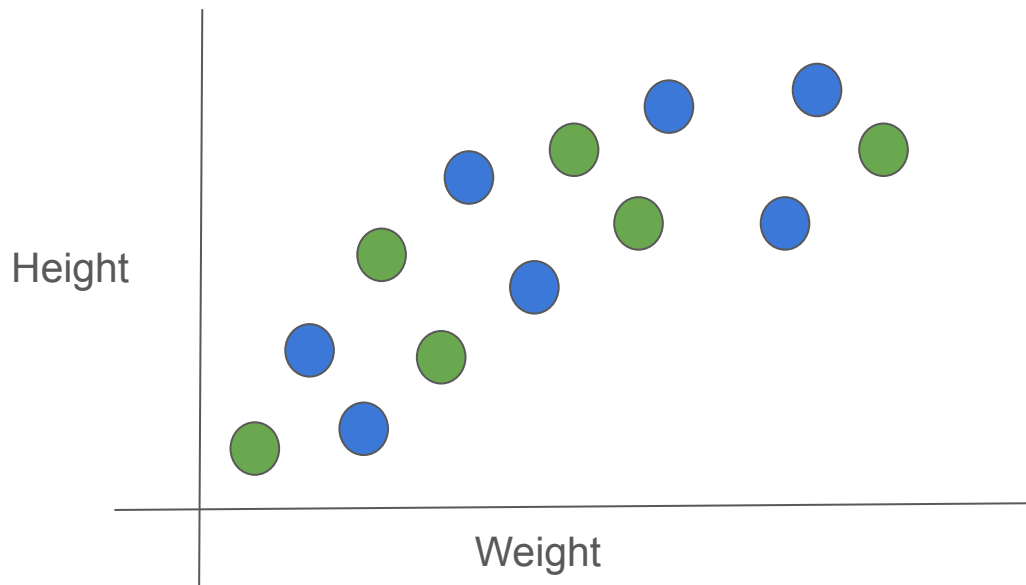
Bias vs Variance

Ideally, we'd like to create a mathematical model that describes the relationship between weight and height



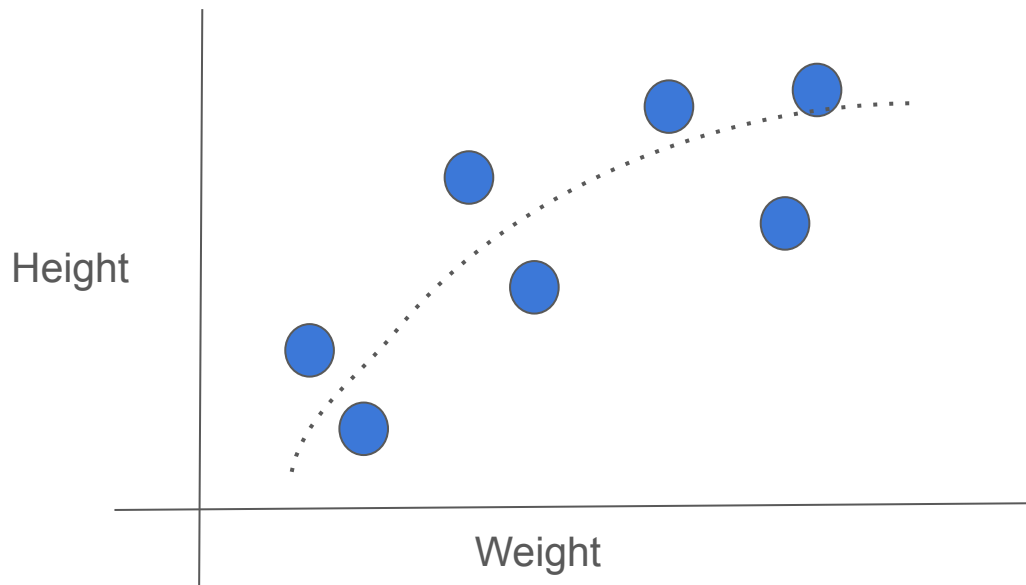
Bias vs Variance

First, let's divide our data into **training** and **testing** data



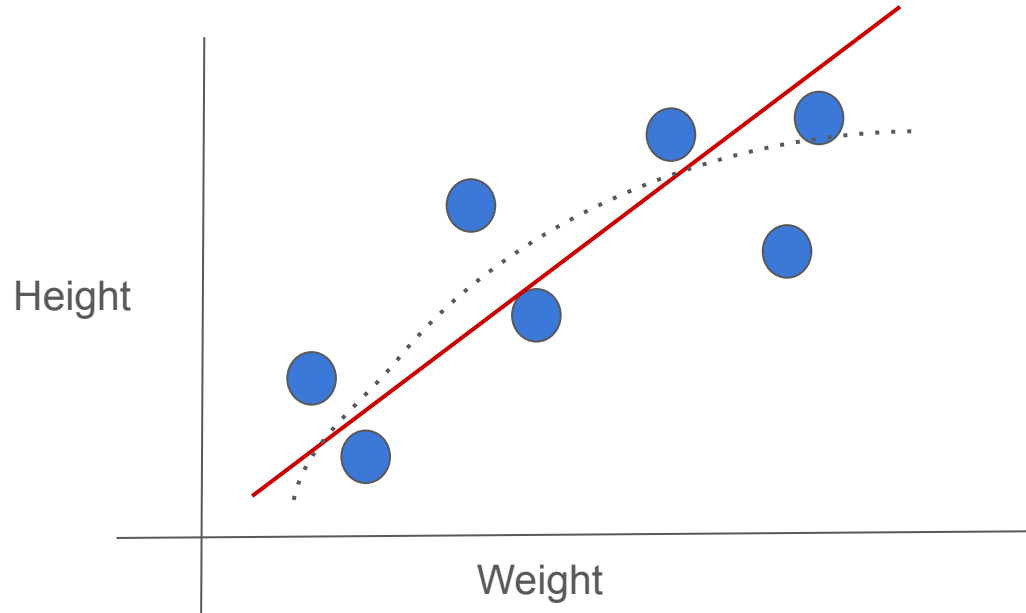
Bias vs Variance

Here are just the training sets



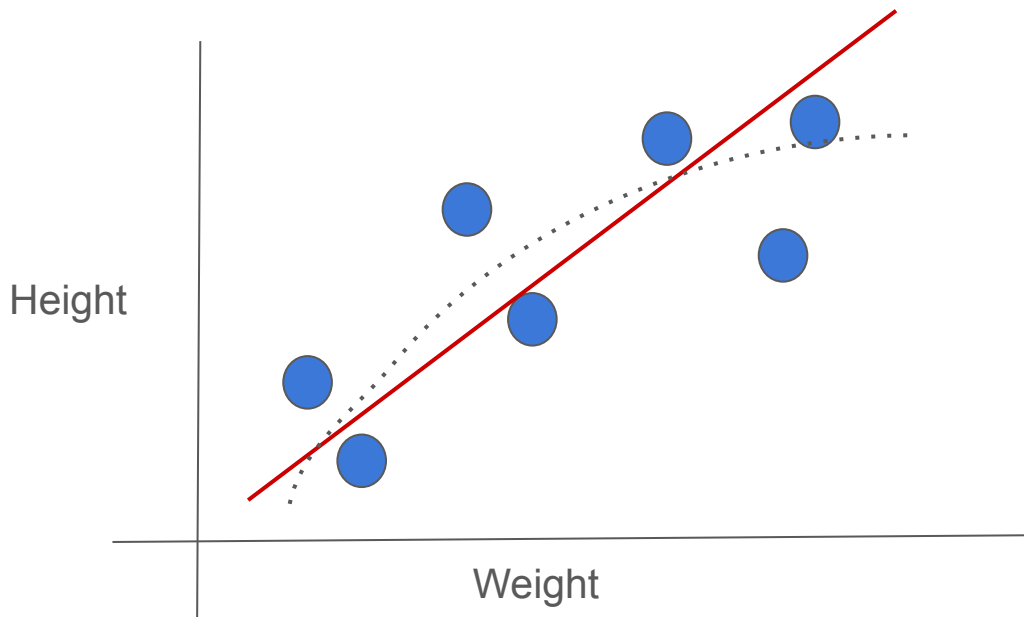
Bias vs Variance

Firstly, lets use linear regression



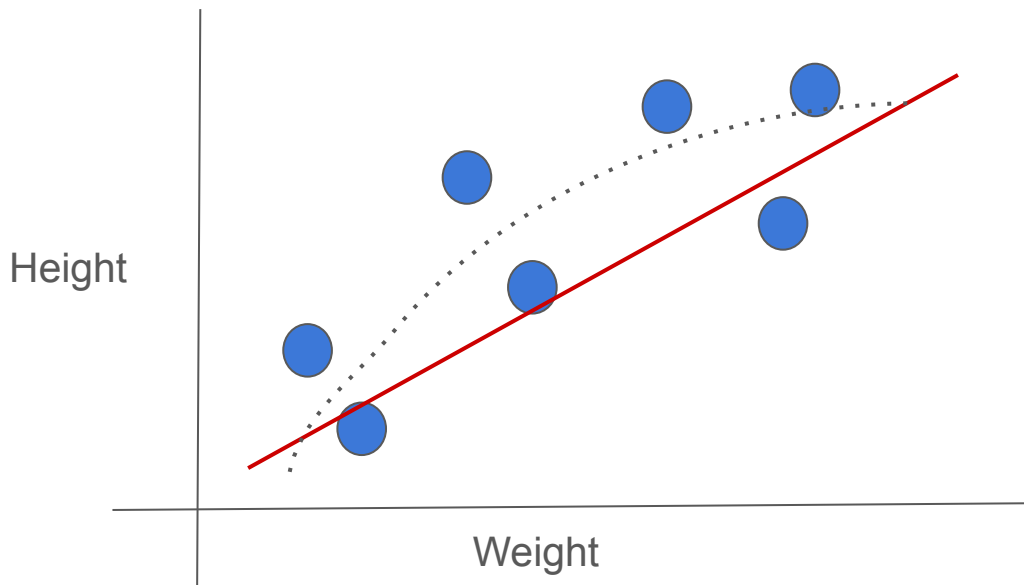
Bias vs Variance

But here's the problem. No matter how we try to fit the line, our linear model never shows us the "true" relationship as it won't curve.



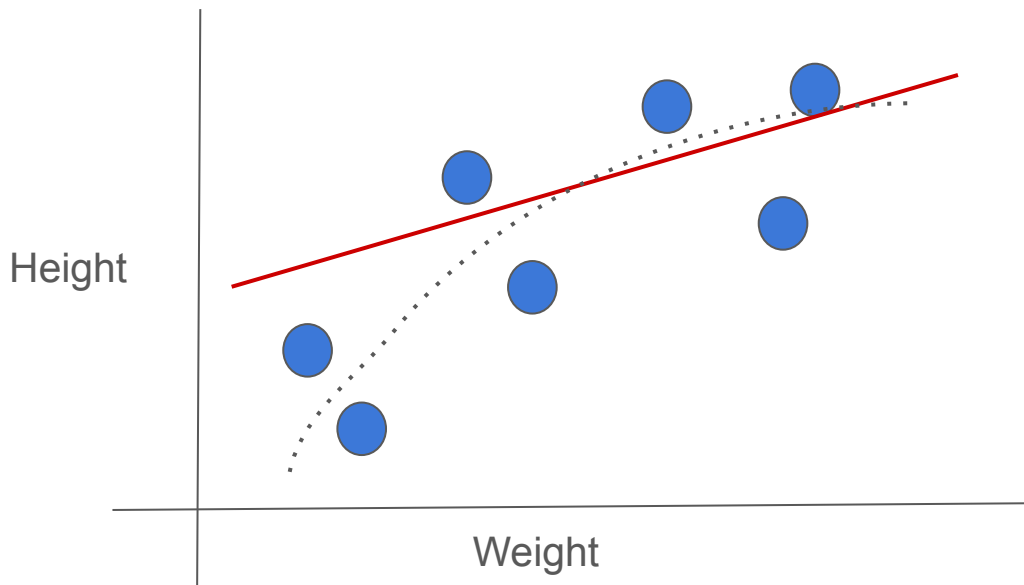
Bias vs Variance

But here's the problem. No matter how we try to fit the line, our linear model never shows us the “true” relationship as it won't curve.



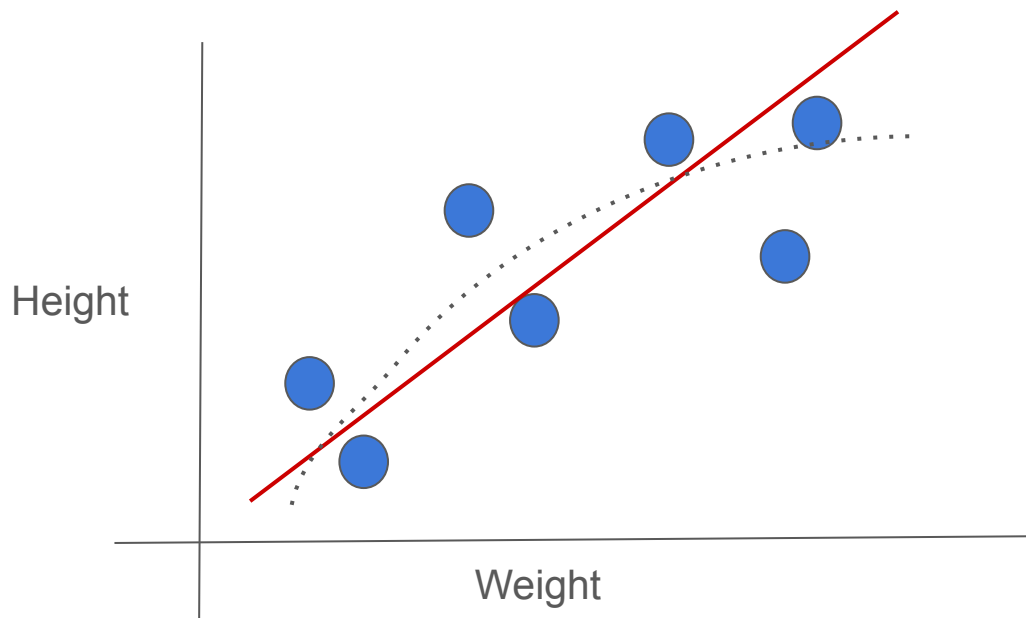
Bias vs Variance

But here's the problem. No matter how we try to fit the line, our linear model never shows us the "true" relationship as it won't curve.



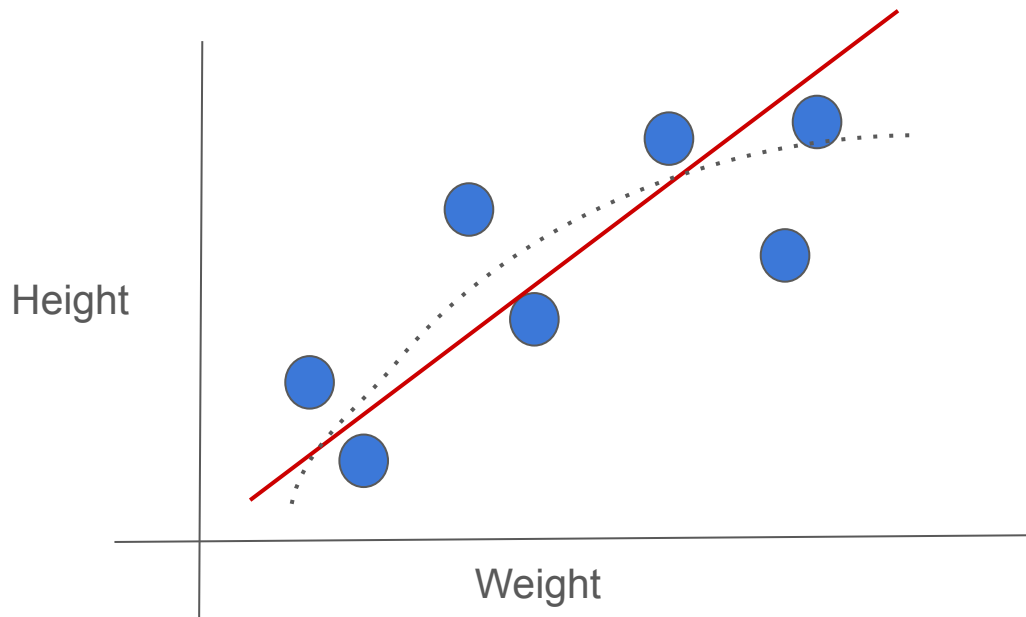
Bias vs Variance

Thus, the straight line will never show us the “true” relationship no matter how well we try to fit.



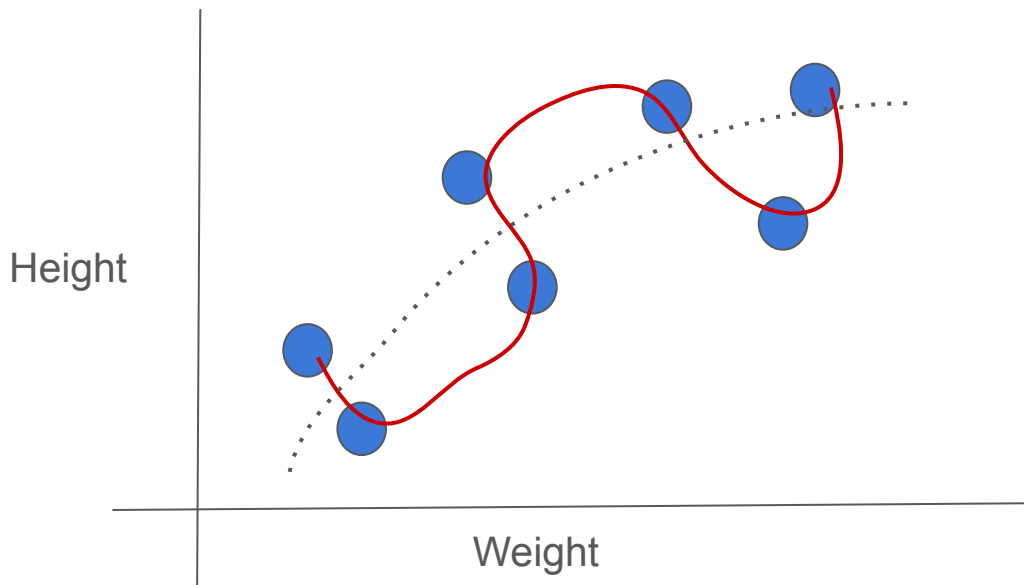
Bias vs Variance

This inability for machine learning model to capture the true relationship is called **bias**. Our straight line model has **high bias**.



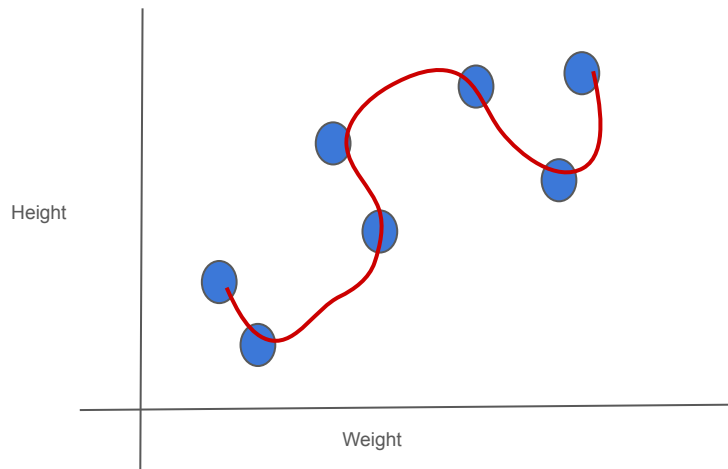
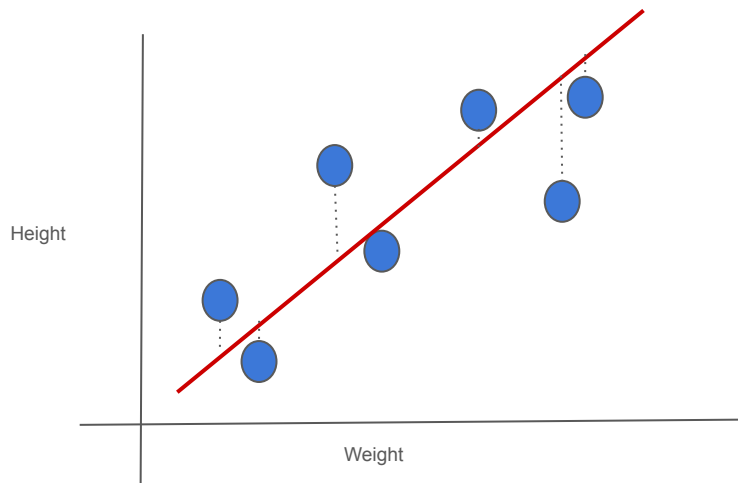
Bias vs Variance

Let's see another example. This time we fit a curvy line.



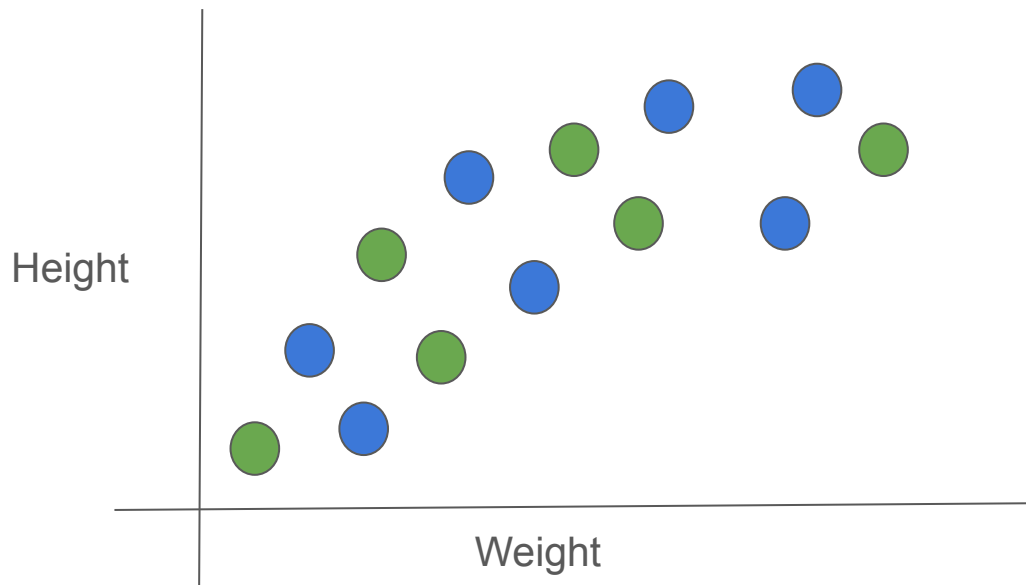
Bias vs Variance

So, when we take **training** set, clearly our curvy line has zero error and the linear model has high error.



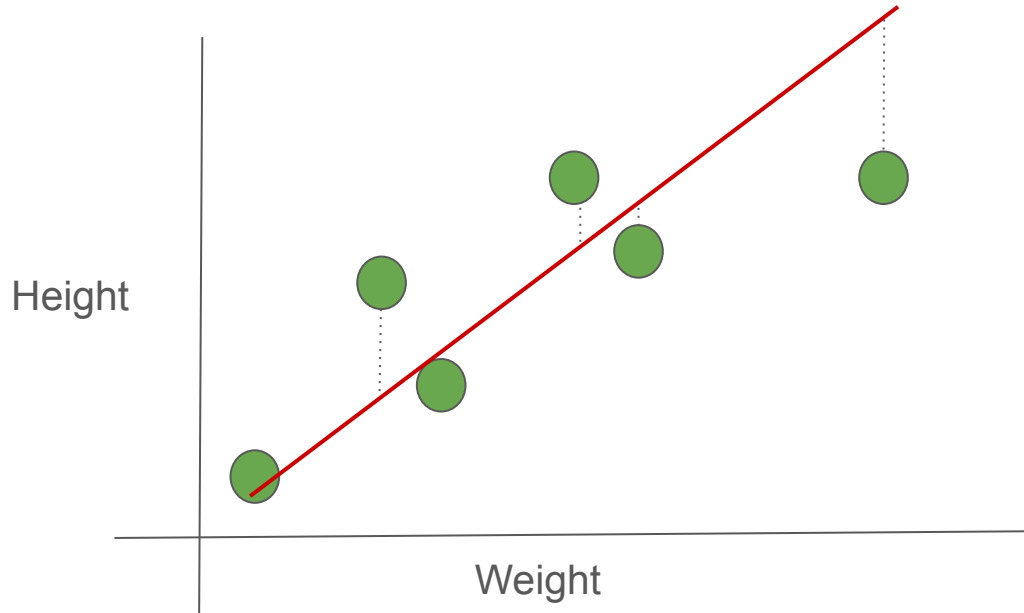
Bias vs Variance

But don't forget we have **test** data too. And when you take only test data into account...



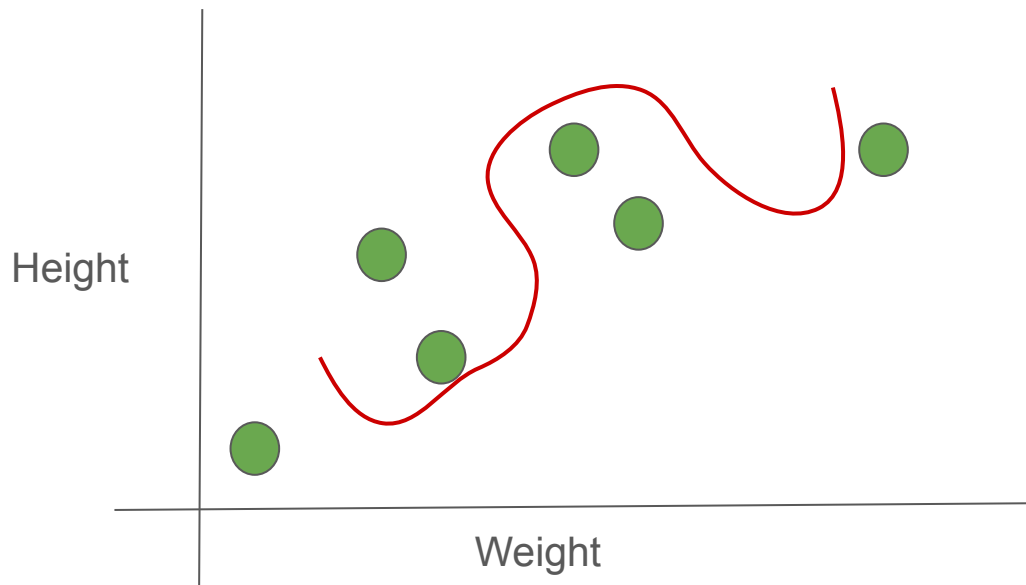
Bias vs Variance

Linear regression model is showing a reasonable inaccuracy



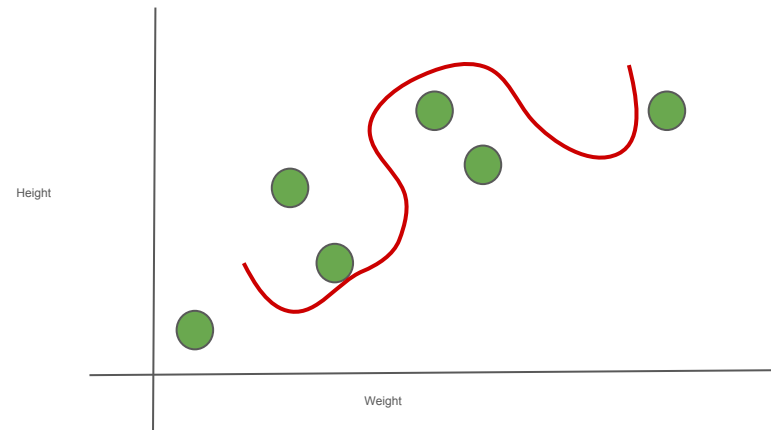
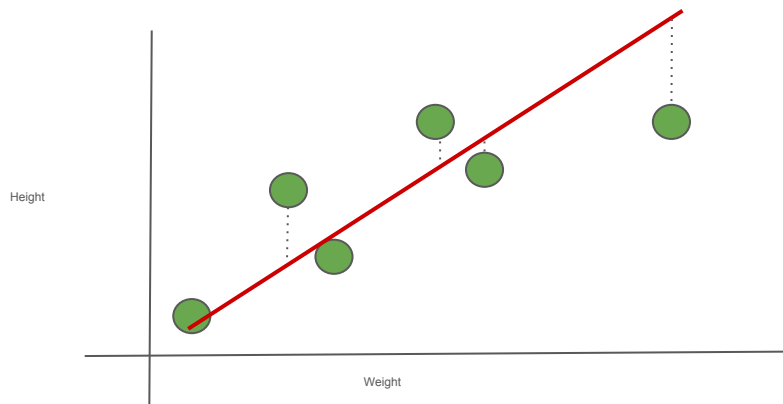
Bias vs Variance

But our curvy line is underperforming.



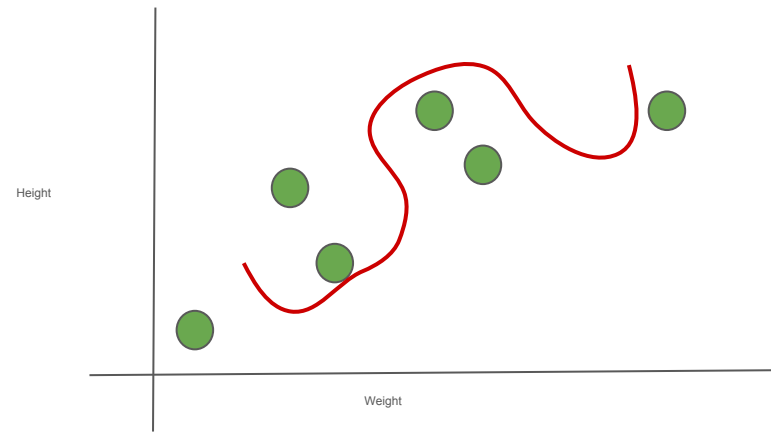
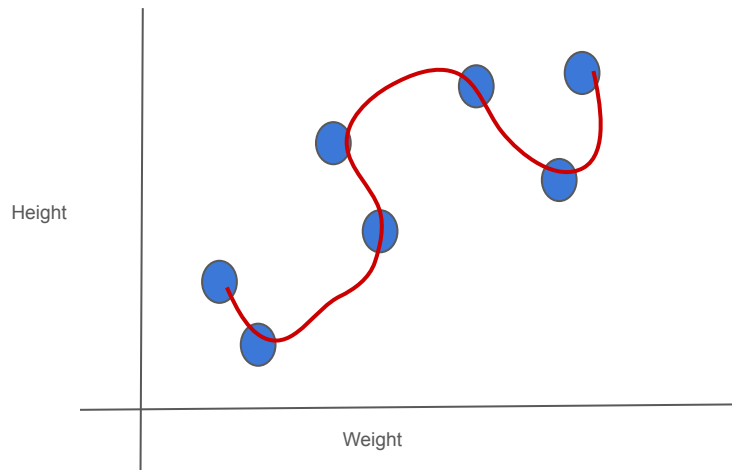
Bias vs Variance

So, when we take **test** set, clearly our curvy line has high error and the linear model has reasonable error like in the case of **training**.



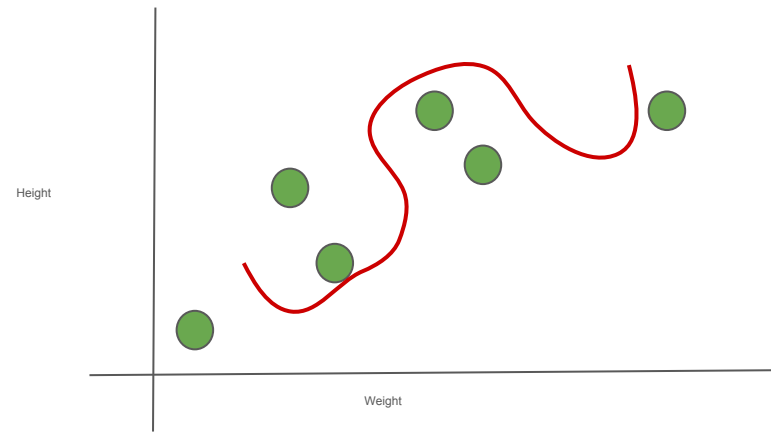
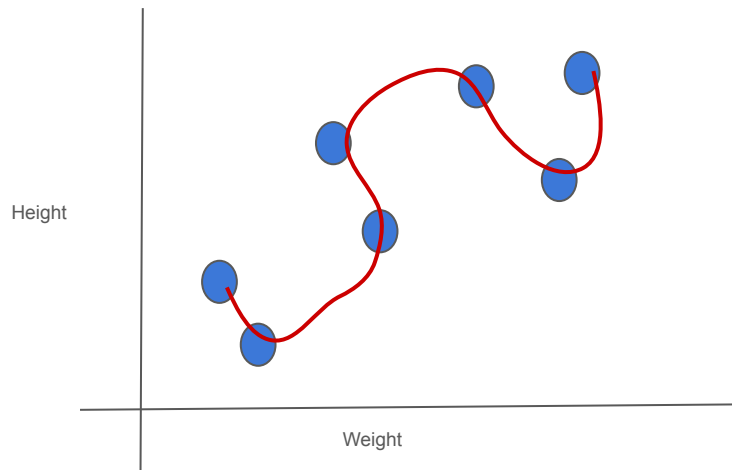
Bias vs Variance

So, even though our curvy line did good job while **training**, the error was too high for **test** set.



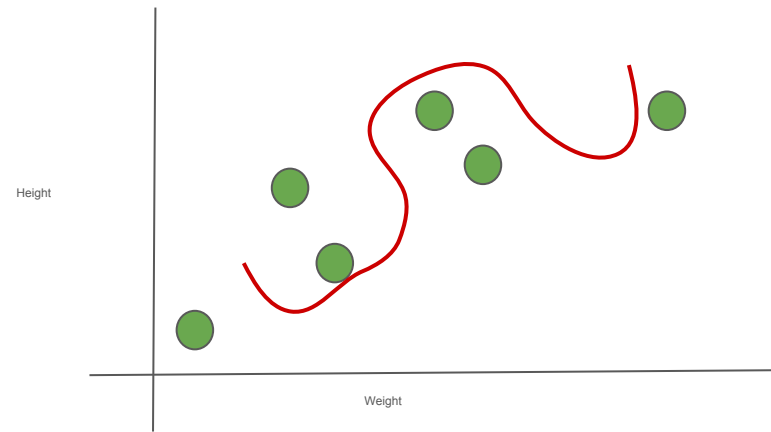
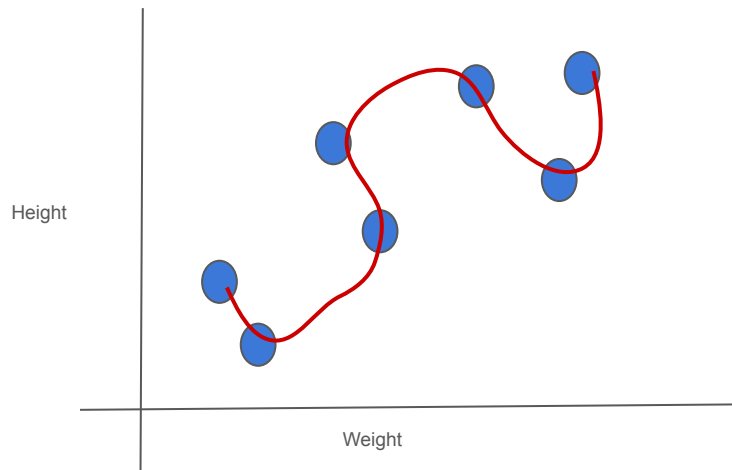
Bias vs Variance

This difference in fits between **test** and **training** set for our model is called variance.



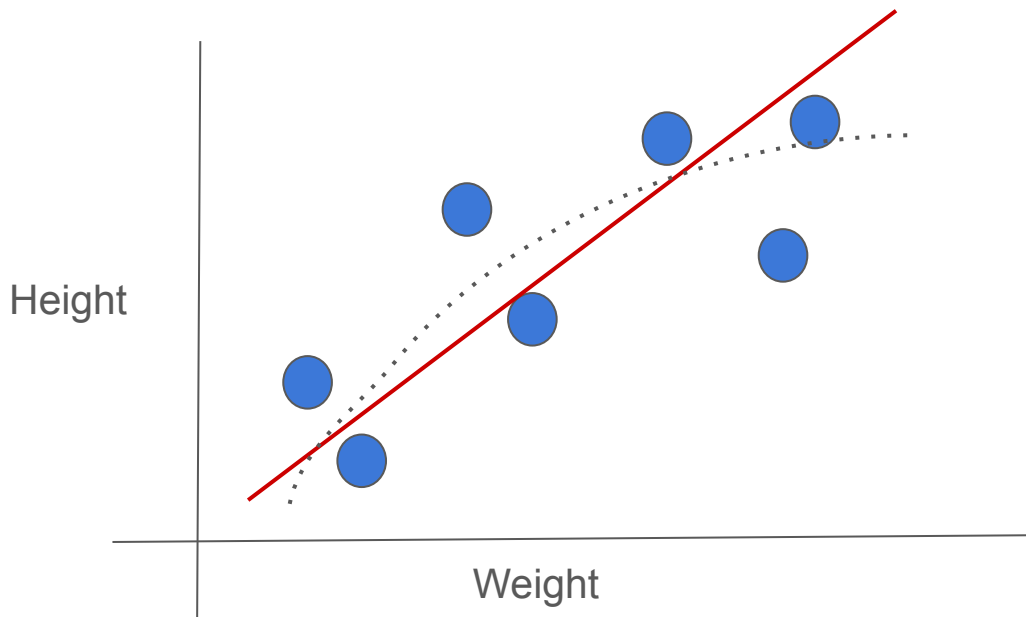
Bias vs Variance

And since our curvy line fits **training** data well and not **testing** data, this also means our model is **overfitting**.



Bias vs Variance

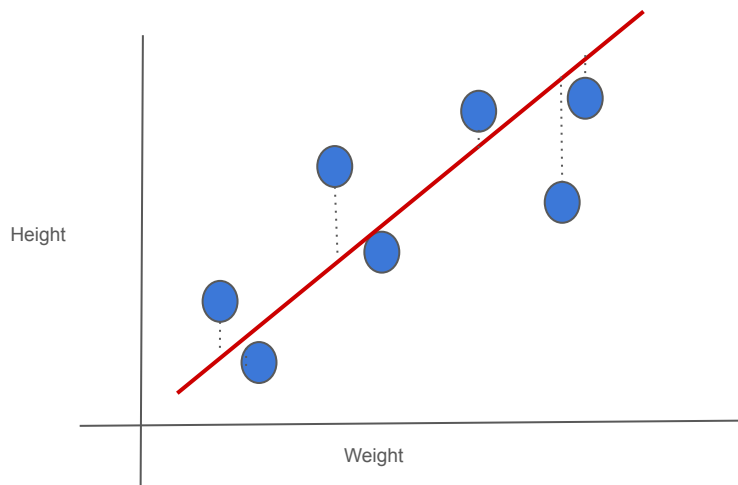
In contrast, since our linear model couldn't generalize well, this situation is called **underfitting**.



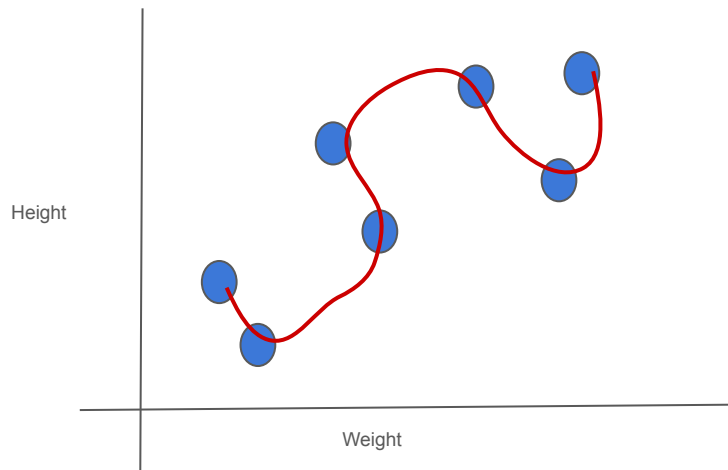
Bias vs Variance

This goal is to find a model that has...

Low variance



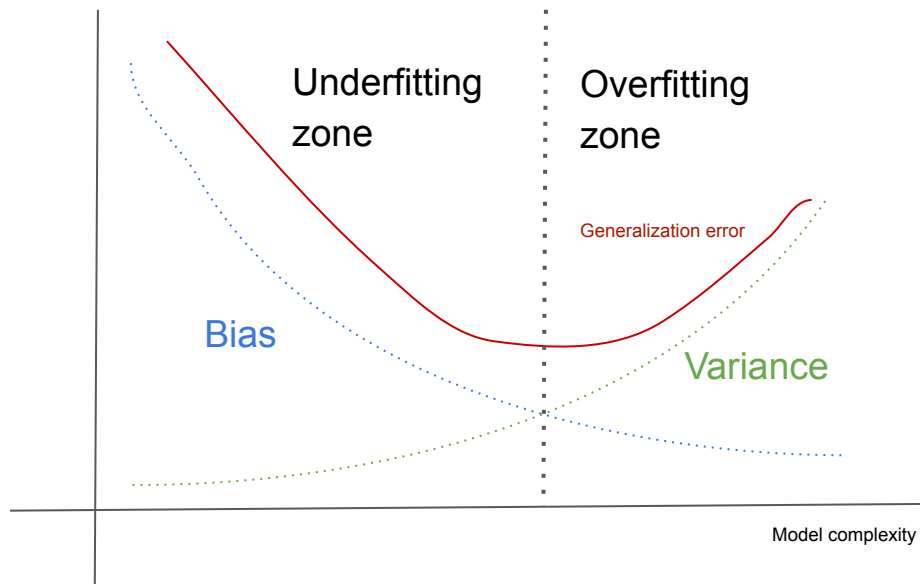
And low bias



Bias Variance Tradeoff

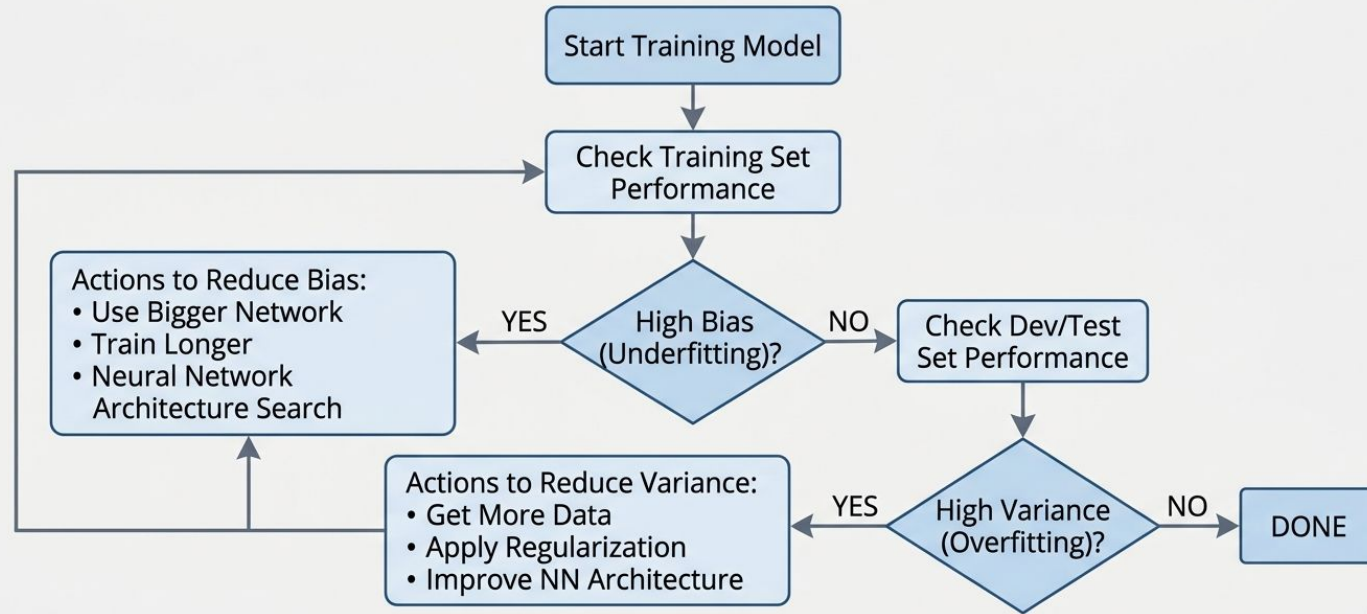
- Models with high bias will have low variance.
- Models with high variance will have a low bias.

The idea is to find a “**sweet spot**”.



Practical workflow of Deep Learning

Model Training Performance Tuning Workflow



Dealing With the Overfitting

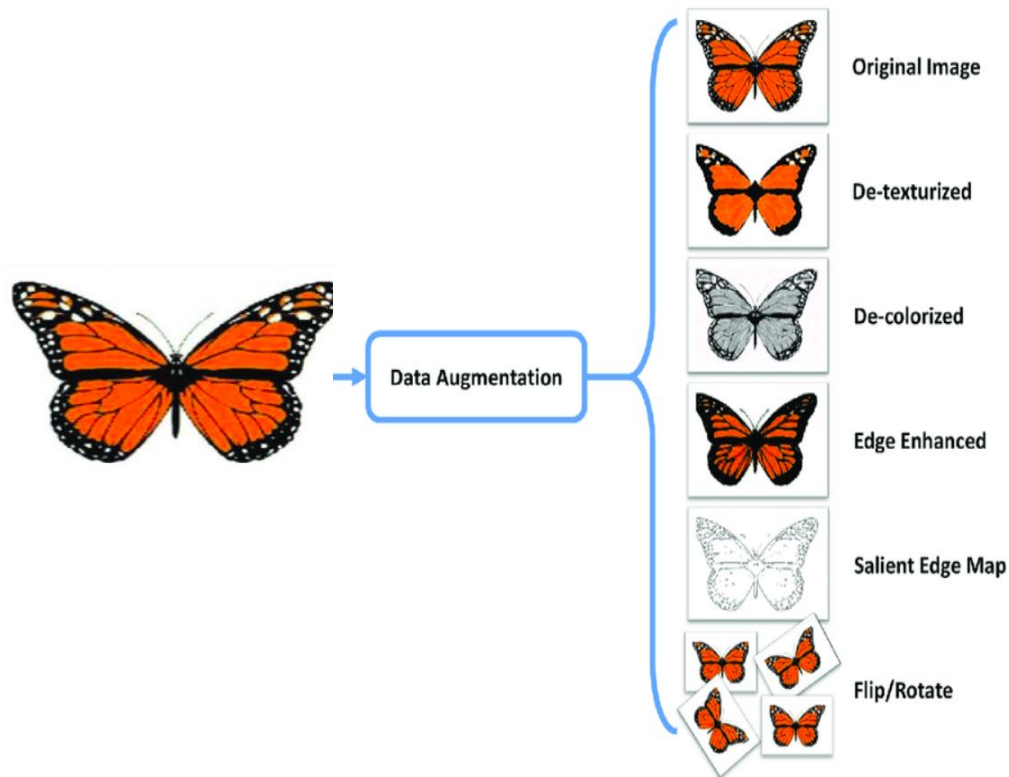
Training With the more Data

- **Include more training data**
 - Simply introducing more data in the training set can remove overfitting of a complicated model as it previously may not have sufficient data to generalize well

But Where to get more data?

Data Augmentation

Technique used in deep learning, especially in computer vision to artificially expand a dataset by creating modified versions of images!!



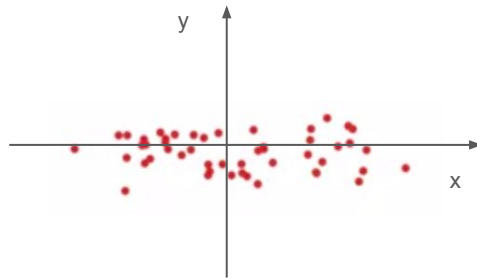
Normalization

- **scaling to a range {min-max Scaling}** $x' = \frac{(x - x_{\text{minimum}})}{x - x_{\text{maximum}}} == \frac{x}{255}$
z-score – Standard Normal Distributions aka standardizations.

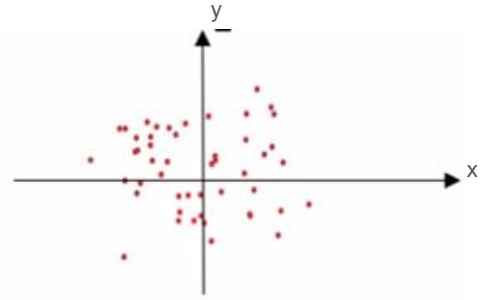
■
$$X' = \frac{x - \text{mean}}{\text{standard deviation}}$$



Mean subtraction



Normalize variance

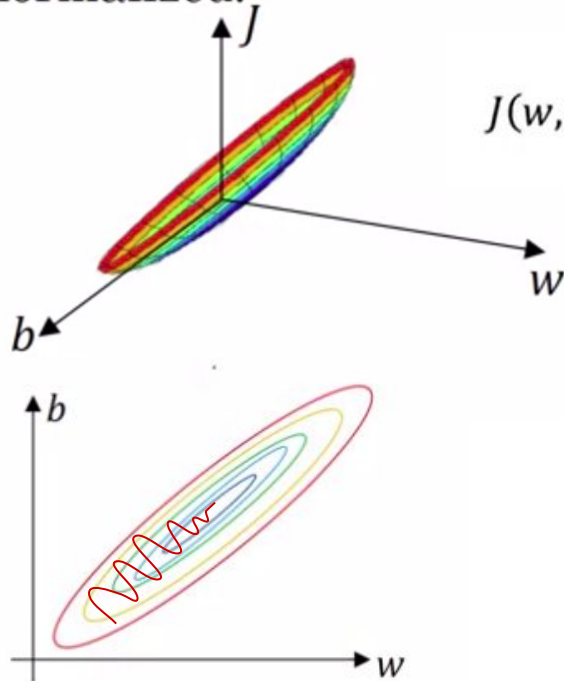


Normalization

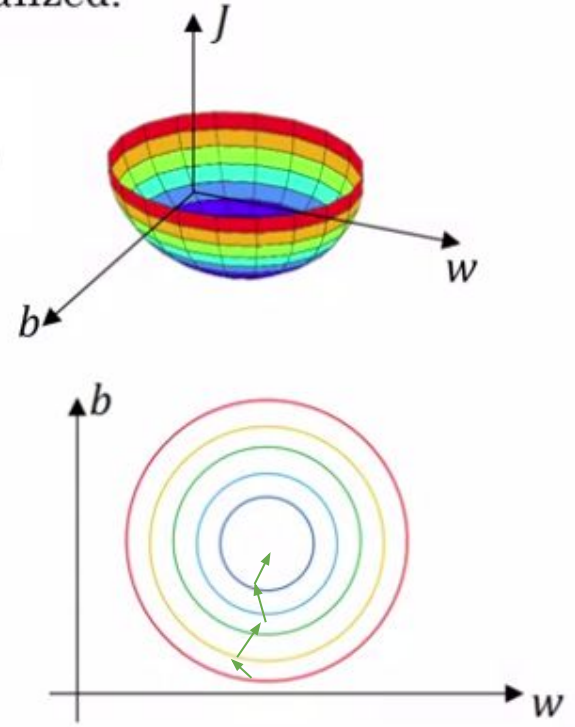
Why?

Normalized:

Unnormalized:

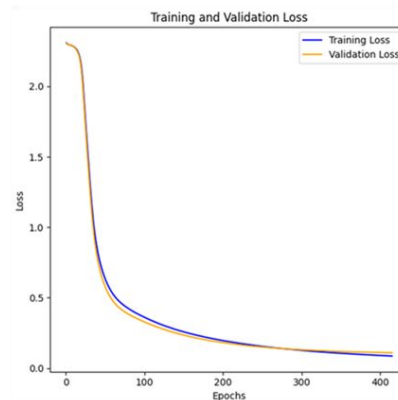
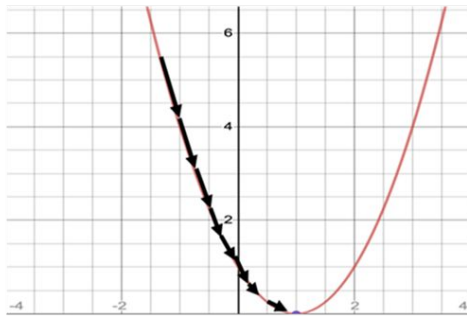
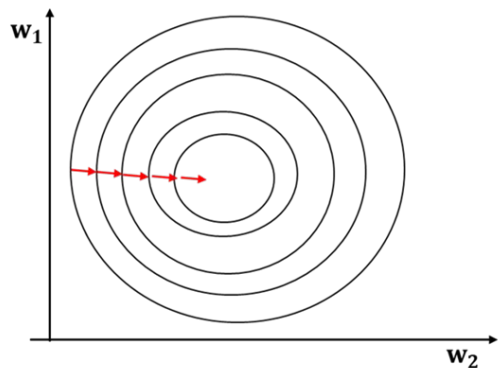


$$J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$



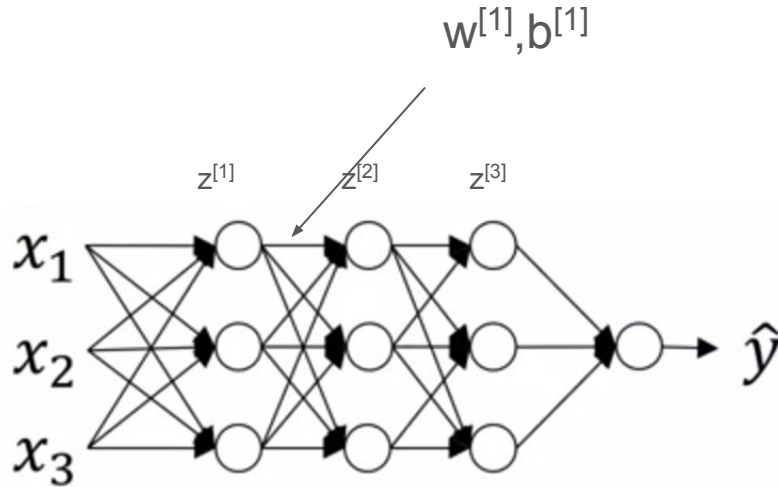
Normalization

Why do we normalize the data in the first place?



- **Q:** However, normalizing the inputs to the network only impacts the first hidden layer.
 - **What about the other hidden layers?**

Batch Normalization

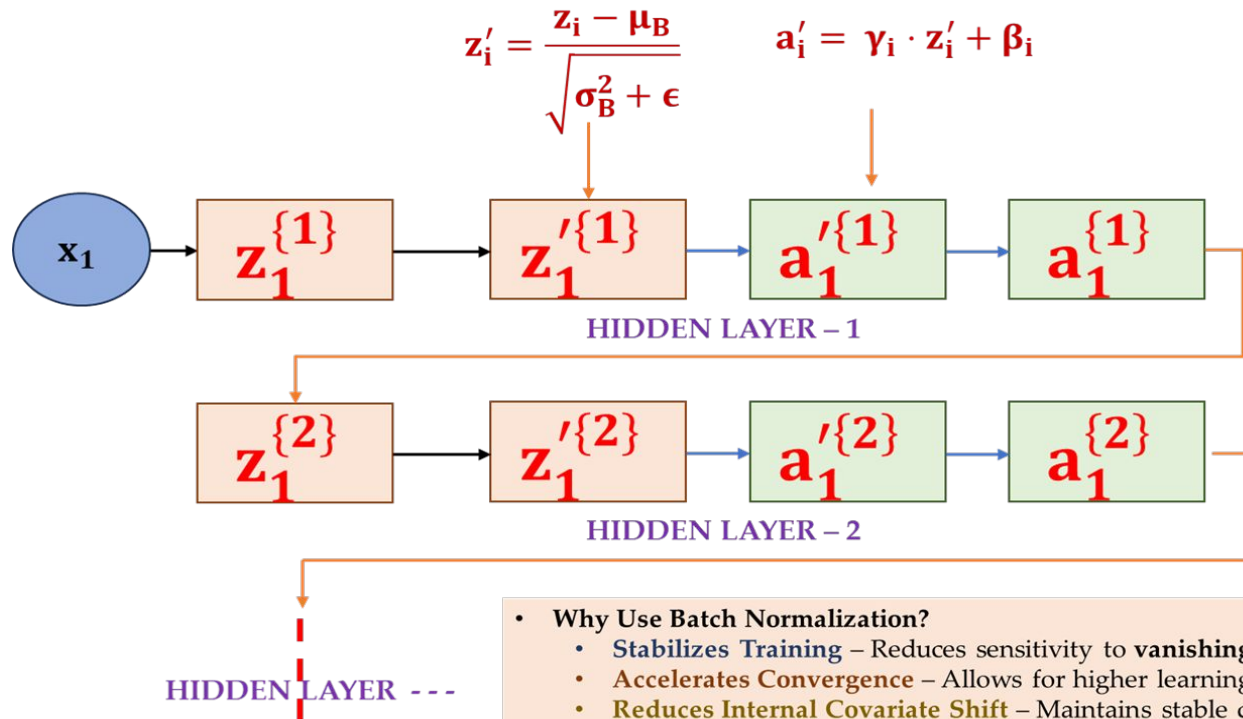


Given some values of Neural net $z^{[1]}, z^{[2]}, \dots, z^{[n]}$

- Compute mean (μ)
- Compute variance (σ^2)
- $z_{\text{norm}}^{[i]} = (z[i] - \mu) / \sqrt{\sigma^2 + \epsilon}$

Can we normalize the values of $z^{[1]}$ in order to train $w^{[1]}, b^{[1]}$ faster?

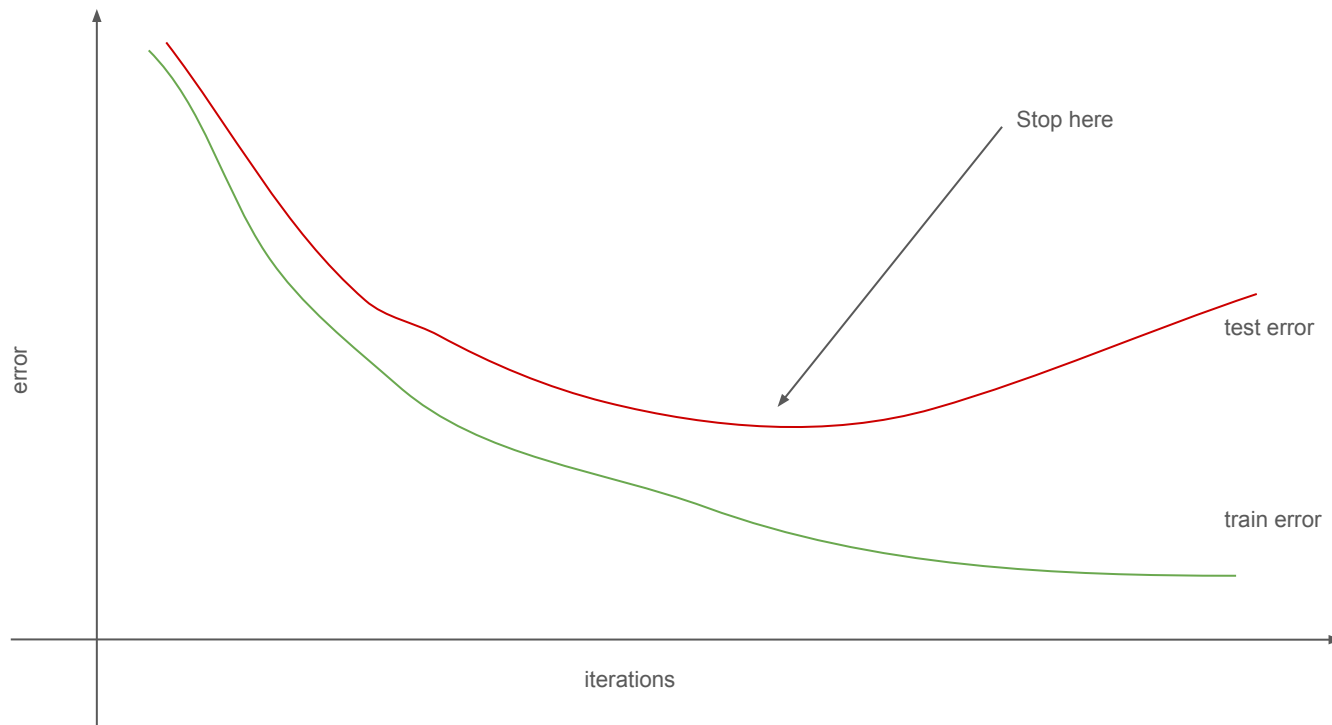
Batch Normalization



- **Why Use Batch Normalization?**

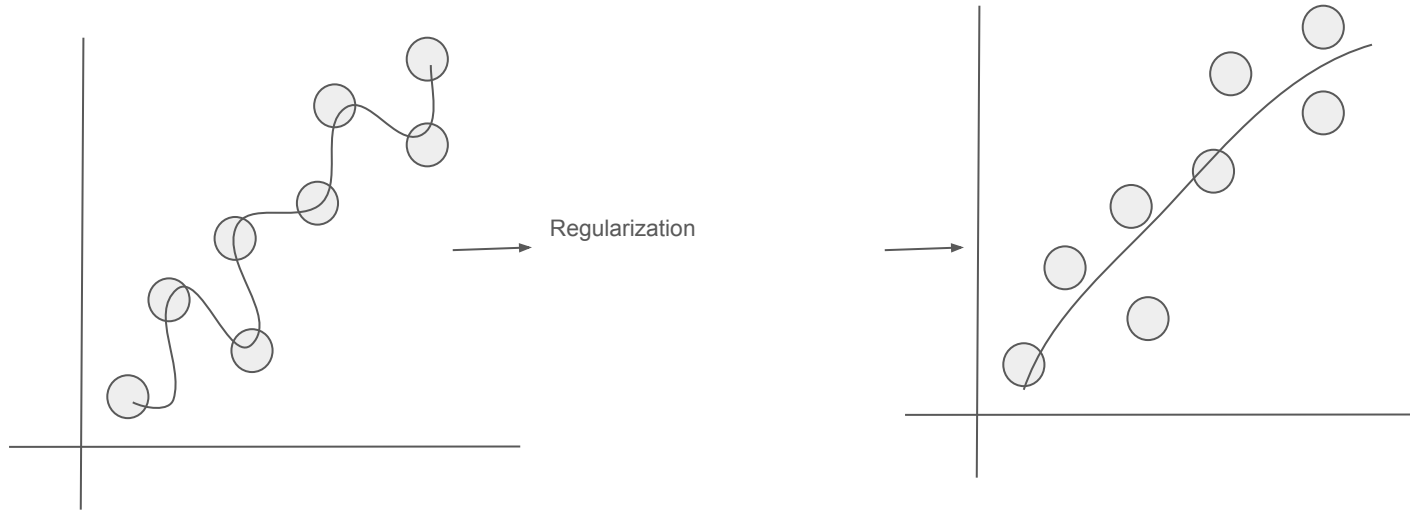
- **Stabilizes Training** – Reduces sensitivity to **vanishing and exploding gradients** (?).
- **Accelerates Convergence** – Allows for higher learning rates.
- **Reduces Internal Covariate Shift** – Maintains stable distributions of activations across layers.
- **Acts as Regularization** – Can reduce the need for **dropout** (?).

Early Stopping



Regularization

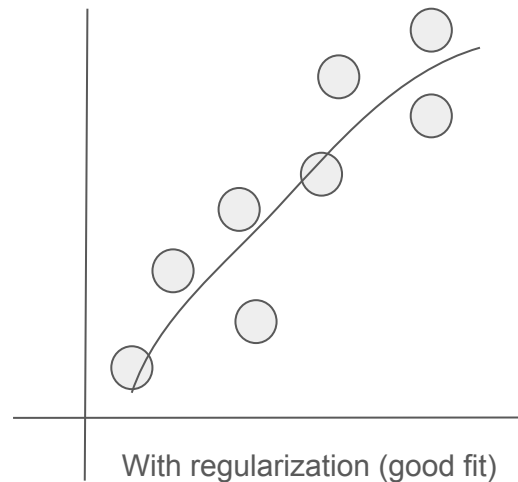
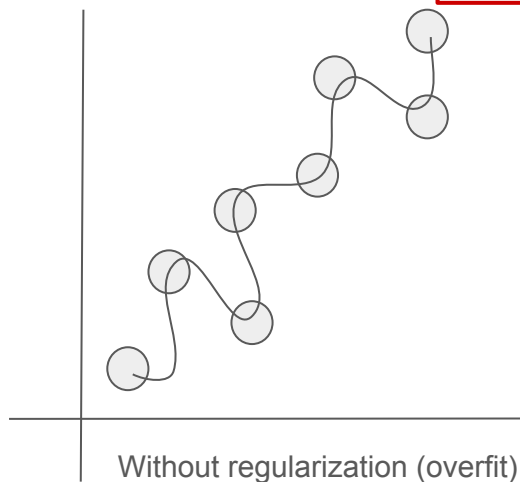
Regularization refers to techniques that are used to calibrate machine learning models in order to minimize the adjusted loss function and prevent overfitting.



Lasso Regularization (L1 Regularization)

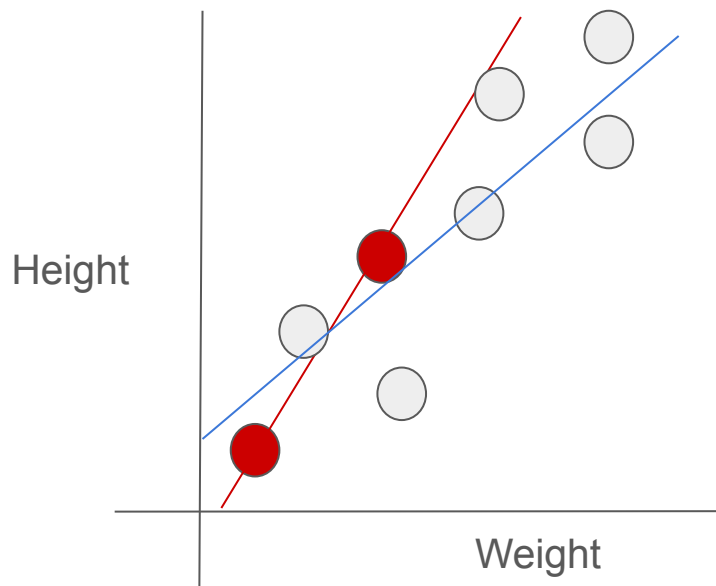
Similar to Ridge Regression i.e. (Loss function + Regularized Term)

$$\mathcal{L}_{\text{reg}}(\boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\theta}) + \lambda \sum_k |\boldsymbol{\theta}_k|$$



Ridge Regularization (L2 Regularization)

Mathematically, we introduce a regularization parameter; which **penalizes** the size of the error.



Loss function + Regularized Term

$$\mathcal{L}_{\text{reg}}(\boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\theta}) + \lambda \sum_k \theta_k^2$$

$\lambda = 0 \Rightarrow$ No impact on the model

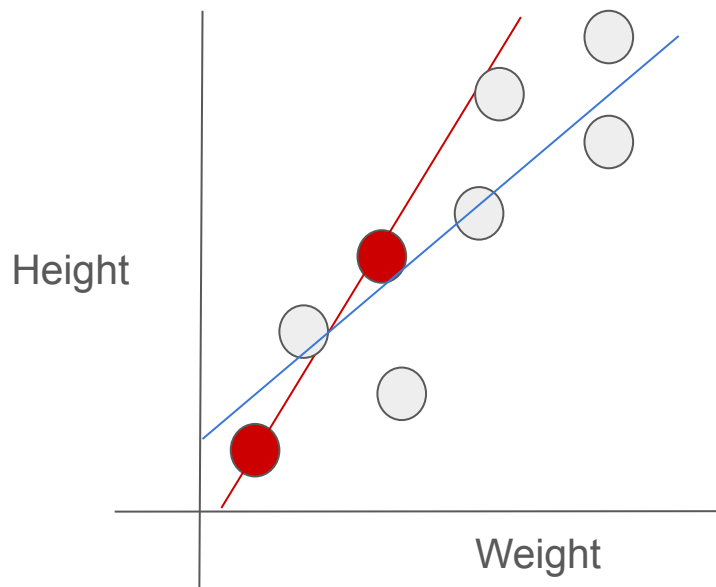
$\lambda = \text{minimal} \Rightarrow$ acceptable model accuracy

$\lambda = \text{high} \Rightarrow$ very high impact/underfitting

λ is chosen by cross-validation and hyperparameter tuning.

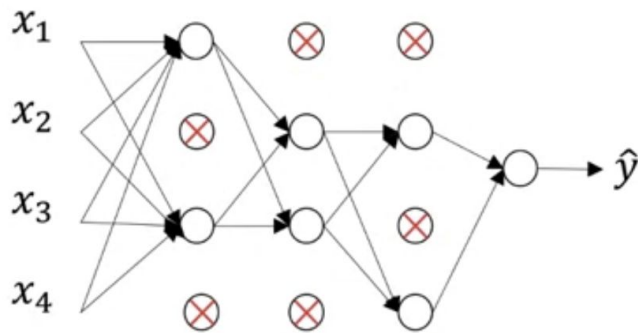
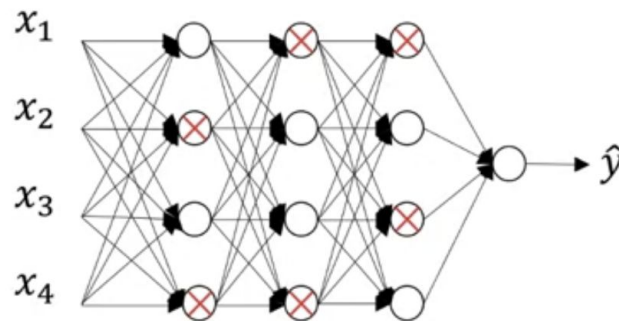
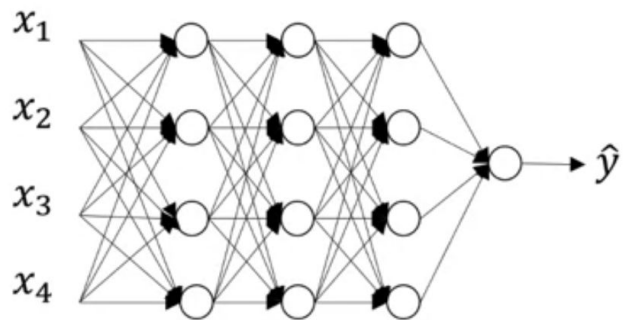
Ridge Regularization (L2 Regularization)

Increase bias slightly, our variance drops significantly.



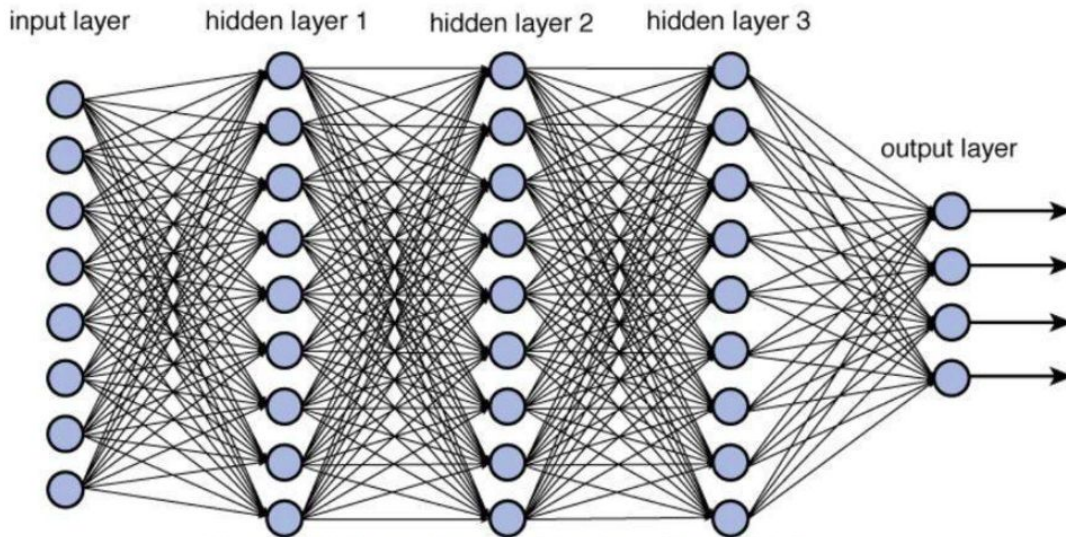
In other words, by starting with slightly worst fit, **Ridge Regularization** can provide better long term predictions.

Dropout



While Training:
Apply a **DropOut layer with $p = 0.5$**

Vanishing and Exploding Gradients



Remember?

$$\delta_j = \frac{\partial E}{\partial e_j} \cdot \phi'(v_j)$$

$$\delta_i = \phi'(v_i) \sum_j \delta_j w_{ji}$$

The model weights may become 0 during training.

The model learns very slowly.

Training stagnates at a very early stage just after a few iterations.

Vanishing and Exploding Gradients

```
regressor1 = tf.keras.models.Sequential([
    tf.keras.layers.Dense(35, input_dim=30, activation='relu',),
    tf.keras.layers.Dense(1, activation='linear')
])
regressor1.compile(loss='mean_squared_error', optimizer=tf.keras.optimizers.Adam())
regressor1.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=30)
regressor1.evaluate(X_test, y_test)
```

Epoch	Progress	Time	loss	val_loss
Epoch 1/30	250/250	1s 2ms/step	loss: nan	val_loss: nan
Epoch 2/30	250/250	0s 1ms/step	loss: nan	val_loss: nan
Epoch 3/30	250/250	0s 1ms/step	loss: nan	val_loss: nan
Epoch 4/30	250/250	0s 1ms/step	loss: nan	val_loss: nan
Epoch 5/30	250/250	0s 1ms/step	loss: nan	val_loss: nan
Epoch 6/30	250/250	0s 1ms/step	loss: nan	val_loss: nan
Epoch 7/30	250/250	0s 1ms/step	loss: nan	val_loss: nan
Epoch 8/30	250/250	0s 1ms/step	loss: nan	val_loss: nan
Epoch 9/30	250/250	0s 1ms/step	loss: nan	val_loss: nan
Epoch 10/30	250/250	0s 1ms/step	loss: nan	val_loss: nan

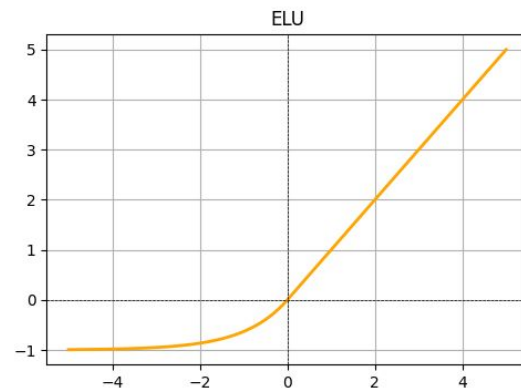
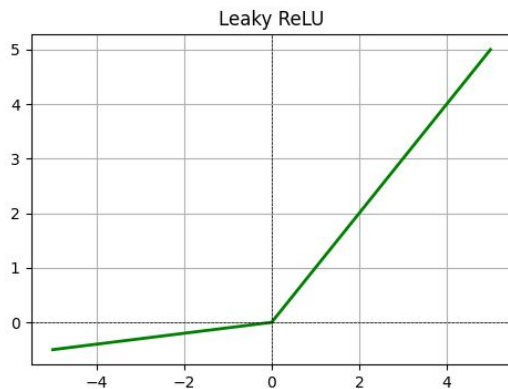
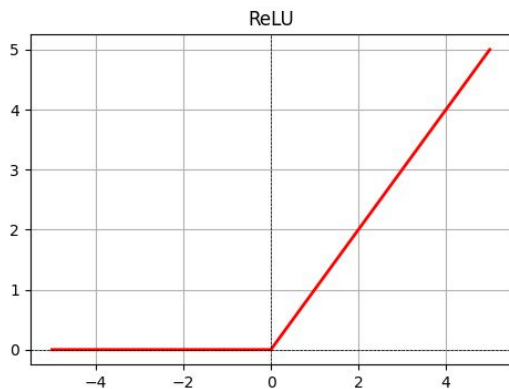
The model weights may become NaN during training.

There is an exponential growth in the model parameter values.

Gradient descent diverges

Choose Activation Function Wisely

- Use ReLU and its variants as activation function

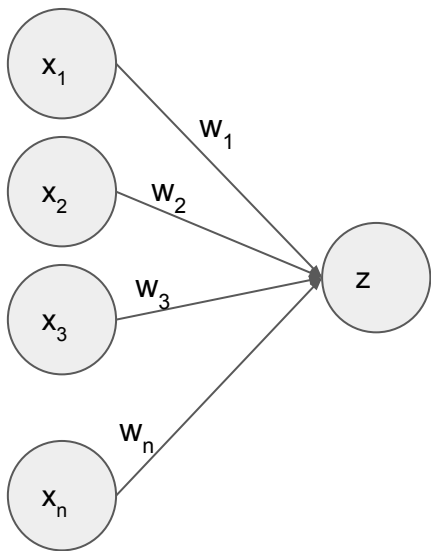


$$f(x) = \begin{cases} x, & \text{if } x > 0 \\ 0, & \text{if } x \leq 0 \end{cases}$$

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}$$

Weight initialization techniques



Avoid zero initialization → Makes the network act like a linear model.

Use random initialization → Prevents neurons from learning the same features.

Issue with random initialization → Can cause vanishing/exploding

In order to regulate z (avoid vanishing or exploding), the larger n is, the smaller you want w_i to be

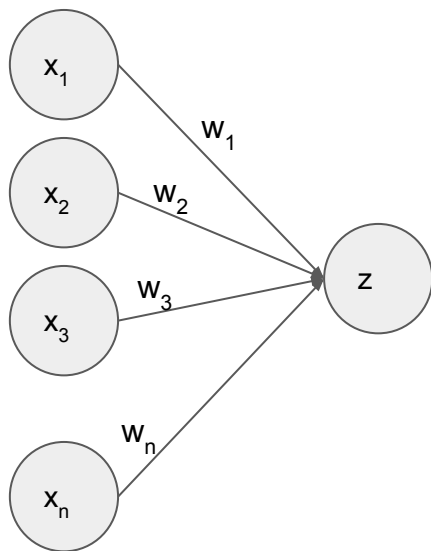
$$z = x_1 w_1 + x_2 w_2 + \dots + x_n w_n$$

$$\text{var}(w_i) = 1/n$$

For layer L

$$W_{[L]} = \text{np.random.randn(shape)} * \text{np.sqrt}(1/n_{[L-1]})$$

Weight initialization techniques



Some Popular Techniques

Xavier Initialization

From Normal Distributions $\rightarrow$

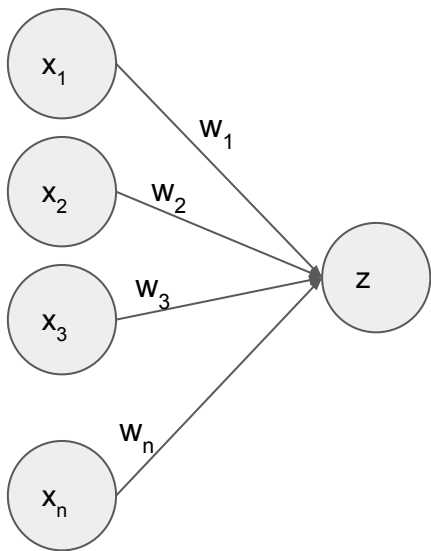
$$\mathbf{W} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right). \text{ Or;}$$

From Uniform Distributions

$$\rightarrow \mathbf{W} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right)$$

Works well for sigmoid and TanH

Weight initialization techniques



Some Popular Techniques

He Initialization for ReLU and its Variants

Weights are initialized as:

- From Normal Distributions:

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right)$$

where n_{in} = number of input neurons.

OR

- For Uniform Distributions:

$$W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{in}}}, \sqrt{\frac{6}{n_{in}}}\right)$$

Mini batch gradient descent

$$X = \begin{bmatrix} \boxed{x^{[1]}} & \boxed{x^{[2]}} & \dots & \boxed{x^{[1000]}} & \boxed{x^{[1001]}} & \dots & \boxed{x^{[m]}} \end{bmatrix}$$

$x\{1\}$ $x\{2\}$ $x\{3\}$ $\dots$ $x\{t\}$

Here, the shape of $X = (n_x, m)$

$$Y = \begin{bmatrix} \boxed{y^{[1]}} & \boxed{y^{[2]}} & \dots & \boxed{y^{[1000]}} & \boxed{y^{[1001]}} & \dots & \boxed{y^{[m]}} \end{bmatrix}$$

$y\{1\}$ $y\{2\}$ $y\{3\}$ $\dots$ $y\{t\}$

Here, the shape of $Y = (1, m)$

If you have 5 million data... let's create batches of 1000 training and testing data

What is the shape of one block of X and Y ?

Mini batch gradient descent

for $t = 1 \dots 5000$ {

forward prop on $X^{\{t\}}$

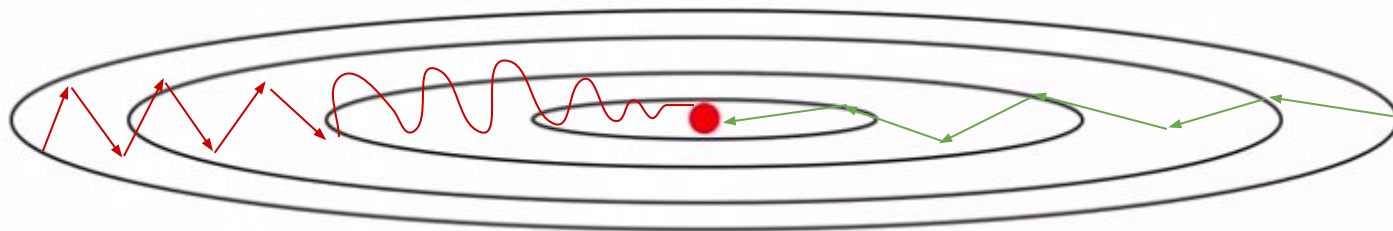
compute cost $J^{\{t\}}$

backprop to compute gradient w.r.t $J^{\{t\}}$

update w and b

} **1 Epoch**

Momentum



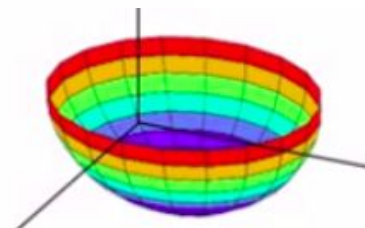
On iteration t :

Compute dW, db on the current mini-batch

$$v_{dW} = \beta v_{dW} + (1 - \beta) dW$$

$$v_{db} = \beta v_{db} + (1 - \beta) db$$

$$W = W - \alpha v_{dW}, \quad b = b - \alpha v_{db}$$



Hyperparameters: α, β

$\beta = 0.9$

RMSprop

On iteration t :

Compute dW, db on the current mini-batch

$$v_{dW} = \beta v_{dW} + (1 - \beta) dW^2$$

$$v_{db} = \beta v_{db} + (1 - \beta) db^2$$

$$W = W - \alpha dW / \sqrt{v_{dW}}, \quad b = b - \alpha db / \sqrt{v_{db}}$$

Hyperparameters: α, β $\beta = 0.9$

Adam

We have,

On iteration **t**:

Compute **dw**, **db** using minibatch

$$v_{dw} = 0, s_{dw} = 0, v_{db} = 0, s_{db} = 0$$

$$\left. \begin{aligned} v_{dw} &= \beta_1 v_{dw} + (1 - \beta_1) dw \\ v_{db} &= \beta_1 v_{db} + (1 - \beta_1) db \end{aligned} \right\} \leftarrow \text{momentum}$$

$$\left. \begin{aligned} s_{dw} &= \beta_2 s_{dw} + (1 - \beta_2) dw^2 \\ s_{db} &= \beta_2 s_{db} + (1 - \beta_2) db^2 \end{aligned} \right\} \leftarrow \text{rms prop}$$

$$\begin{aligned} v_{dw}^{corrected} &= v_{dw} / (1 - \beta_1^t) \\ v_{db}^{corrected} &= v_{db} / (1 - \beta_1^t) \end{aligned}$$

$$\begin{aligned} s_{dw}^{corrected} &= s_{dw} / (1 - \beta_2^t) \\ s_{db}^{corrected} &= s_{db} / (1 - \beta_2^t) \end{aligned}$$

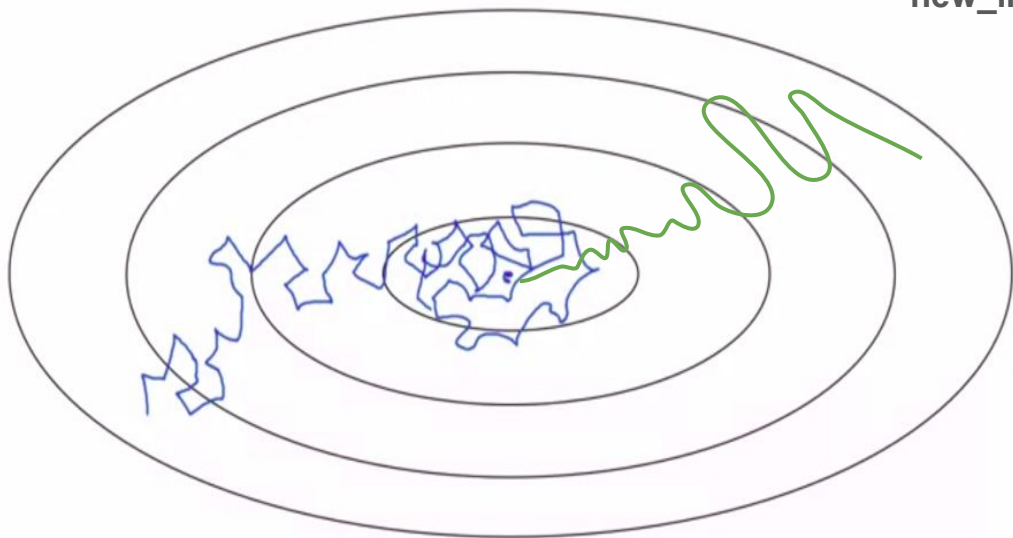
$$w = w - \alpha \frac{v_{dw}^{corrected}}{\sqrt{s_{dw}^{corrected} + \epsilon}}$$

$$b = b - \alpha \frac{v_{db}^{corrected}}{\sqrt{s_{db}^{corrected} + \epsilon}}$$

Learning Rate Decay

The idea is to slowly decay the learning rate so that convergence occur.

$$\text{new_lr} = \text{current_lr} / (1 + \text{decay_rate} * \text{epoch_num})$$



Hyperparameter Tuning

α

β (0.9 appx)

$\beta_1, \beta_2, \epsilon$ (0.9, 0.99, 10^{-8} respectively)

Number of layers

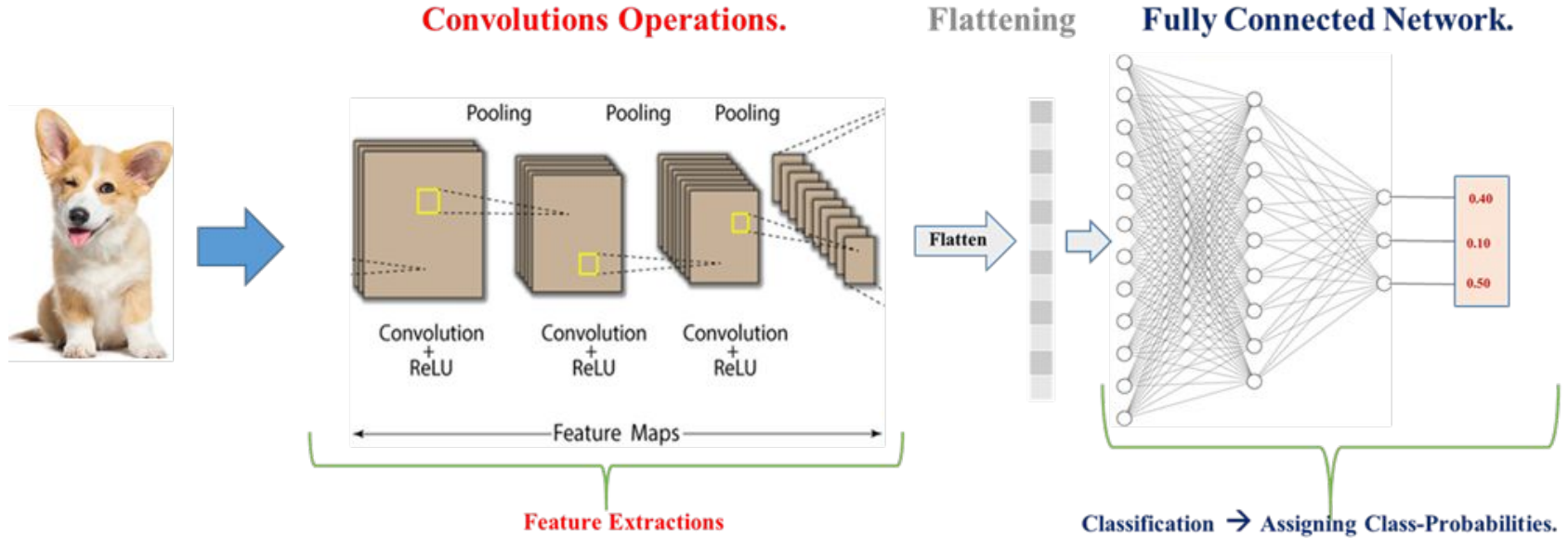
Number of hidden units

Learning rate decay

Mini-batch size

Some Popular CNN Architectures

Recap: CNN Architecture



An Architecture

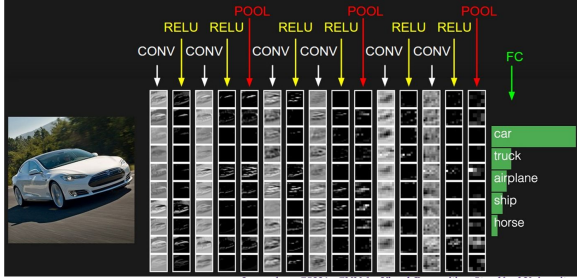


Fig: An example architecture:
 ([Input] → CONV(ReLU) → CONV(ReLU) → Pool → CONV(ReLU) → CONV(ReLU) → Pool → CONV(ReLU) → CONV(ReLU) → Pool → Flatten → FCN → [Class Scores])

A ConvNet architecture is in the simplest case a list of Layers that transform the image volume into an output volume (holding the class scores).

- There are a few foundational layer in every ConvNet architecture which are **{CONV(ReLU), Pool, FCN}**.
- Each layer accepts an
 - input 3D volume and transforms it to an output 3D volume through a differentiable function.
 - **[Height × Width × Channel]**
- Each layer may or may not have parameters.
 - E.g. CoNV and FCN do, ReLU and Pool don't.
- Each layer may or may not have additional hyperparameters.
 - E.g. CoNV, Pool, FCN does, ReLU doesn't
- Fully Connected Layer will compute the class scores, resulting the volume of size
 - **[1 × 1 × Number of Classes]**

Imagenet Dataset

contains over 14 million labeled images across 1,000 categories



About ImageNet Competition

- The **ImageNet Large Scale Visual Recognition Challenge (ILVRC)** was a highly influential annual competition in computer vision, running from 2010 to 2017.
- It served as a benchmark for evaluating the performance of image classification and object detection models on the ImageNet dataset
- **Impact of ImageNet Competition:**
 - **Accelerated deep learning research:** Led to CNN dominance in vision tasks.
 - **Improved AI applications:** Helped power self-driving cars, medical imaging, facial recognition, and more.
 - **Drove industry adoption:** Companies like Google, Facebook, and Nvidia heavily invested in AI.
 - **It ended in 2017**, because the error rates dropped below human – level performance.

ImageNet Major Breakthrough.

Major Breakthrough in ILSVRC

Year	Model	Top ~ 5 Error %	Key Innovations
2010	Traditional Methods	28.2%	Handcrafted features
2012	AlexNet	16.4%	Deep CNNs, ReLU activation, DropOut
2013	ZFNet	11.2%	Optimized AlexNet with better filters
2014	VGG - 16	7.3%	Deep networks (16 – 19 layers)
2014	GoogleLeNet (Inception v1)	6.7%	Inception Module
2015	ResNet – 152	3.6%	Deep residual learning (skip connections)
2016	Inception – v4 and ResNet	3.1%	Improved depth and efficiency
2017	SENet	2.3%	Squeeze and Extraction blocks for attention.

Note: Human – level performance is estimated at ~5% error.

AlexNet.

- **AlexNet** is a deep convolutional neural network (CNN) architecture that won the **ImageNet Large Scale Visual Recognition Challenge (ILSVRC) in 2012**.
 - It was developed by **Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton** and marked a breakthrough in deep learning for computer vision.
- **Key Features:**
 - **8 Layers Deep:** 5 Convolutional Layers + 3 Fully Connected Layers.
 - **ReLU activation:** Used instead of sigmoid to speed up training and mitigate the vanishing gradient problem.
 - **DropOut Regularization:** Applied in fully connected layers to prevent overfitting.
 - **Overlapping Max – Pooling:** Helps reduce spatial dimensions while maintaining important features.
 - **Data Augmentation:** Techniques like random cropping, flipping and color jittering were used to improve generalizations.
 - **Trained with GPU Acceleration:** AlexNet was one of the first deep networks trained on two NVIDIA GPUs to handle large – scale computations efficiently.

AlexNet Architecture Breakdown.



Layer	Type	Details
1	Conv + ReLU	96 filters (11×11), stride 4
2	Max Pooling	3×3, stride 2
3	Conv + ReLU	256 filters (5×5), stride 1
4	Max Pooling	3×3, stride 2
5	Conv + ReLU	384 filters (3×3), stride 1
6	Conv + ReLU	384 filters (3×3), stride 1
7	Conv + ReLU	256 filters (3×3), stride 1
8	Max Pooling	3×3, stride 2
9	Fully Connected	4096 neurons + ReLU + Dropout
10	Fully Connected	4096 neurons + ReLU + Dropout
11	Fully Connected	1000 neurons (softmax output)

AlexNet Parameters.

Parameters Calculation for Each layer.

Layer	Type	Filters/Neurons	Kernel Size	Parameters Calculation	Total Parameters
Conv1	Conv2D	96	$11 \times 11 \times 3$	$(11 \times 11 \times 3 \times 96) + 96$	34,944
Conv2	Conv2D	256	$5 \times 5 \times 96$	$(5 \times 5 \times 96 \times 256) + 256$	614,656
Conv3	Conv2D	384	$3 \times 3 \times 256$	$(3 \times 3 \times 256 \times 384) + 384$	885,120
Conv4	Conv2D	384	$3 \times 3 \times 384$	$(3 \times 3 \times 384 \times 384) + 384$	1,327,488
Conv5	Conv2D	256	$3 \times 3 \times 384$	$(3 \times 3 \times 384 \times 256) + 256$	884,992
FC6	Fully Connected	4096	-	$(6 \times 6 \times 256 \times 4096) + 4096$	37,752,832
FC7	Fully Connected	4096	-	$(4096 \times 4096) + 4096$	16,781,312
FC8	Fully Connected	1000	-	$(4096 \times 1000) + 1000$	4,097,000
Total					≈ 60 Mil

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 55, 55, 96)	34,944
max_pooling2d (MaxPooling2D)	(None, 27, 27, 96)	0
conv2d_1 (Conv2D)	(None, 27, 27, 256)	614,656
max_pooling2d_1 (MaxPooling2D)	(None, 13, 13, 256)	0
conv2d_2 (Conv2D)	(None, 13, 13, 384)	885,120
conv2d_3 (Conv2D)	(None, 13, 13, 384)	1,327,488
conv2d_4 (Conv2D)	(None, 13, 13, 256)	884,992
max_pooling2d_2 (MaxPooling2D)	(None, 6, 6, 256)	0
flatten (Flatten)	(None, 9216)	0
dense (Dense)	(None, 4096)	37,752,832
dropout (Dropout)	(None, 4096)	0
dense_1 (Dense)	(None, 4096)	16,781,312
dropout_1 (Dropout)	(None, 4096)	0
dense_2 (Dense)	(None, 1000)	4,097,000

Total params: 62,378,344 (237.95 MB)

Trainable params: 62,378,344 (237.95 MB)

Non-trainable params: 0 (0.00 B)

VGG 16

- VGG16 is a deep convolutional neural network (CNN) that emphasizes simplicity by using very small convolutional filters (3x3) stacked on top of each other.
 - It primarily uses **convolutional layers** followed by **fully connected (FC) layers** for classification.
 - Not any key new features, a clean trick to build deeper network with cleaner architecture.
- Here's the breakdown:
- **Feature Extraction:**
 - **Convolutional Layers:**
 - VGG 16 is composed of **13 convolutional layers**.
 - Each of these layers uses **3 × 3 filters (with a stride of 1 and padding of 1)** to process the input image.
 - The use of small filters ensures that the **receptive field gradually increases as you go deeper into the network**, allowing the network to capture **increasingly complex features**.
 - There are also max – pooling layers (**typically 2 × 2 pooling with stride of 2**) placed after certain Convolutional layers to reduce the spatial dimensions of the feature maps and prevent overfitting.

VGG 16

- **Classification:**

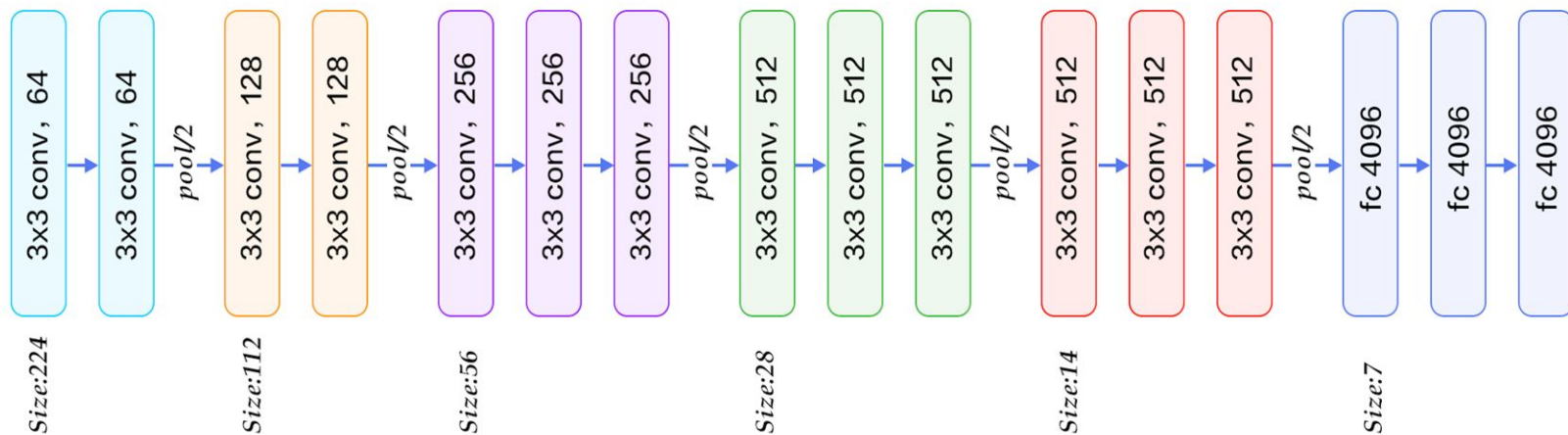
- **Fully Connected (FC) Layer:**

- After passing through the convolutional and pooling layers, the output is flattened into a 1D vector.
 - The flattened vector is passed through three fully connected layers.
 - The **first two FC layers each have 4096 neurons**,
 - The output of the final FC layer is passed into a softmax layer.

- **Final FC layer - Softmax Layer:**

- 1000 softmax neurons as ImageNet has 1000 different classes.
 - The softmax function is applied to the output of the last fully connected layer. This convnets the network's output into a probability distribution across the classes.
 - The class with the highest probability is selected as the predicted label.

VGG – 16 Architecture.



VGG – 16 Parameters.

Feature Extraction.

Layer	Type	Filters/Neurons	Kernel Size	Parameters Calculation	Total Parameters
Conv1_1	Conv2D	64	3x3	$(3 \times 3 \times 3 \times 64) + 64$	1,792
Conv1_2	Conv2D	64	3x3	$(3 \times 3 \times 64 \times 64) + 64$	36,928
MaxPool1	MaxPooling2D	-	2x2	-	0
Conv2_1	Conv2D	128	3x3	$(3 \times 3 \times 64 \times 128) + 128$	73,856
Conv2_2	Conv2D	128	3x3	$(3 \times 3 \times 128 \times 128) + 128$	147,584
MaxPool2	MaxPooling2D	-	2x2	-	0
Conv3_1	Conv2D	256	3x3	$(3 \times 3 \times 128 \times 256) + 256$	295,168
Conv3_2	Conv2D	256	3x3	$(3 \times 3 \times 256 \times 256) + 256$	590,080
Conv3_3	Conv2D	256	3x3	$(3 \times 3 \times 256 \times 256) + 256$	590,080
MaxPool3	MaxPooling2D	-	2x2	-	0
Conv4_1	Conv2D	512	3x3	$(3 \times 3 \times 256 \times 512) + 512$	1,180,160
Conv4_2	Conv2D	512	3x3	$(3 \times 3 \times 512 \times 512) + 512$	2,359,808
Conv4_3	Conv2D	512	3x3	$(3 \times 3 \times 512 \times 512) + 512$	2,359,808
MaxPool4	MaxPooling2D	-	2x2	-	0
Conv5_1	Conv2D	512	3x3	$(3 \times 3 \times 512 \times 512) + 512$	2,359,808
Conv5_2	Conv2D	512	3x3	$(3 \times 3 \times 512 \times 512) + 512$	2,359,808
Conv5_3	Conv2D	512	3x3	$(3 \times 3 \times 512 \times 512) + 512$	2,359,808
MaxPool5	MaxPooling2D	-	2x2	-	0

Classification.

Layer	Type	Filters/Neurons	Kernel Size	Parameters Calculation	Total Parameters
Flatten	Flatten	-	-	-	0
FC1	Fully Connected	4096	-	$(7 \times 7 \times 512 \times 4096) + 4096$	102,764,544
FC2	Fully Connected	4096	-	$(4096 \times 4096) + 4096$	16,781,312
FC3	Fully Connected	1000	-	$(4096 \times 1000) + 1000$	4,097,000
Total				Total Parameters	138,357,544

Google LeNet – Inceptions V1.

- **GoogLeNet**, also known as **Inception v1**, was the winning model of the **ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2014**.
- It achieved a **top-5 error rate of 6.7%**, significantly improving over previous architectures like **AlexNet** and **VGG-16**, while using far fewer parameters.
- **Key Innovations in Inception v1:**
 - **Inception Modules:**
 - Introduced a novel multi scale feature extraction approach.
 - Instead of using a single kernel size (like **VGG** or **AlexNet**), it used **1×1 ; 3×3 ; and 5×5** convolutions in **parallel**.
 - This allowed the network to capture fine details (small kernels) and broader patterns (large kernel) at the same time.
 - **1×1 Convolutions for Dimensionality Reduction:**
 - Used 1×1 convolutions before **3×3 ; and 5×5** convolutions to reduce feature map depth reducing computational cost significantly.

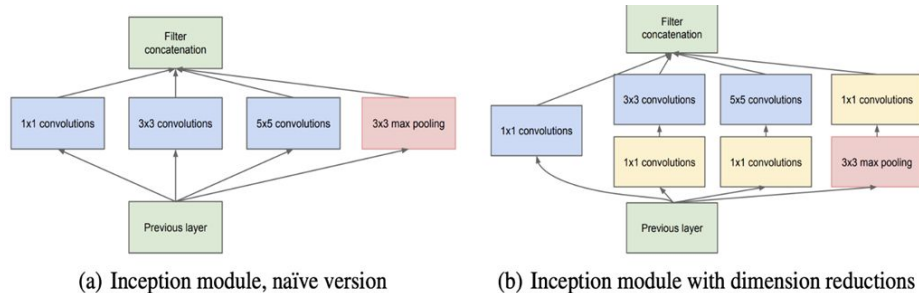
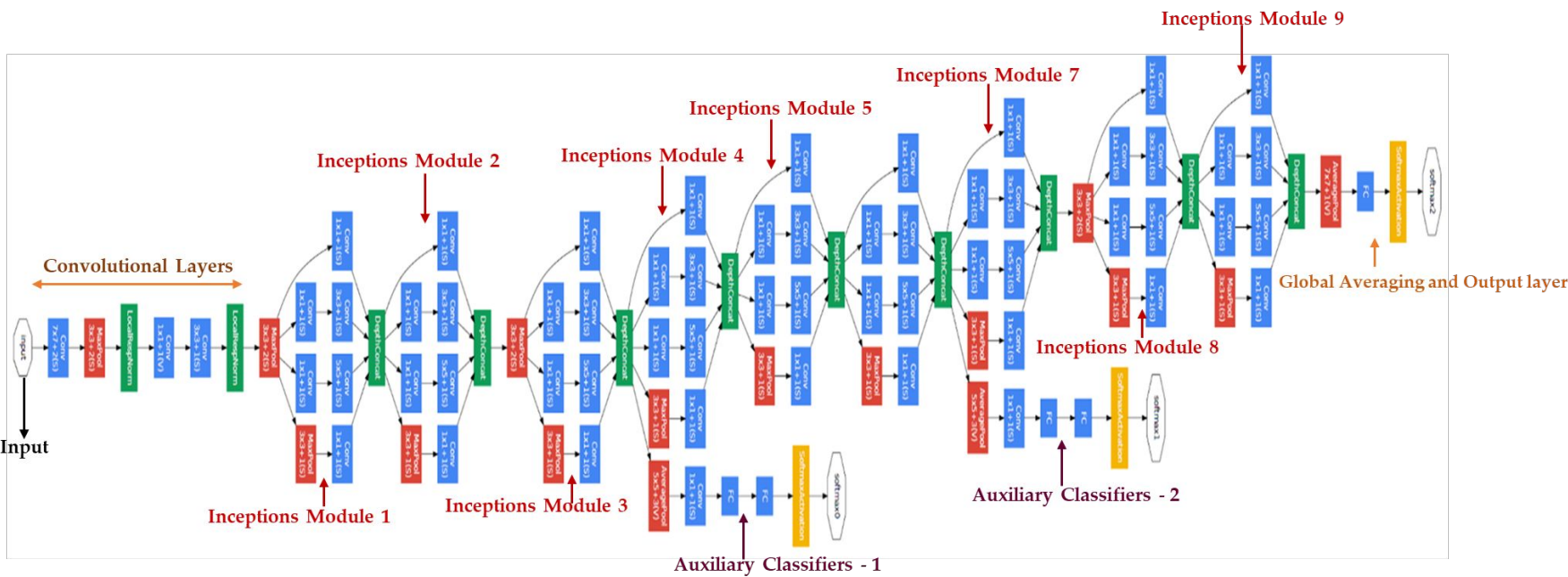


Figure 2: Inception module

Google LeNet – Inceptions V1.

- **Key Innovations in Inception v1 (Contd.):**
 - **Deeper Network with Fewer Parameters:**
 - 22 layers deep compared to AlexNet (8 layers) and VGG – 16 (16 layers).
 - Used only 6 million parameters, compared to AlexNet's 60 million parameter.
 - **Global Average Pooling Instead of Fully Connected Layers:**
 - Replaced traditional fully connected layers with average pooling, reducing parameters and overfitting.
 - **Auxiliary Classifiers (Extra Loss Branches):**
 - Added two smaller classifier branches at intermediate layers to help gradient flow, reducing the vanishing gradient problem in deep networks.

Google LeNet – Inceptions V1 - Architecture.



ResNet (Residual Networks)

- **ResNet** (Kaiming He et. al 2015 winner.) introduces the concept of **residual connections** or **skip connections**, which help mitigate the vanishing gradient problem and enable training of deeper networks.
- Residual or Skip Connections:
 - **Layer inputs** are **propagated** and **added** to the **layer output**.
 - Mitigate the problem of vanishing gradients during training
 - Allow training very deep NN (**with over 1,000 layers**)
 - Several ResNet variants exist: 18, 34, 50, 101, 152, and 200 layers
 - Are used as base models of other state-of-the-art NNs.

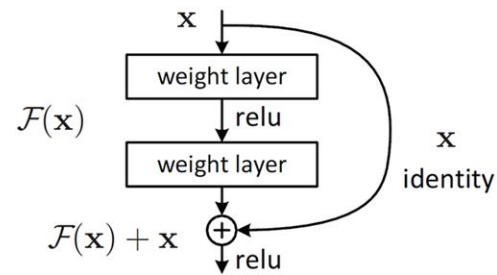


Fig: Skip Connections.

Workings of ResNet.

- **Feature Extraction:**
 - **Convolutional Layers:**
 - Like VGG16, ResNet uses convolutional layers to extract features from the input image, but it also includes **residual blocks**.
 - **Residual Block:**
 - A **residual block** consists of two or more convolutional layers,
 - with a **shortcut or skip connection** that bypasses these layers and **adds the input to the output**.
 - This helps the model learn the **residual** mapping
 - the difference between the input and the output; instead of the direct mapping, which aids in training very deep networks.
 - ResNet uses **batch normalization** after each convolution to stabilize learning and speed up convergence.

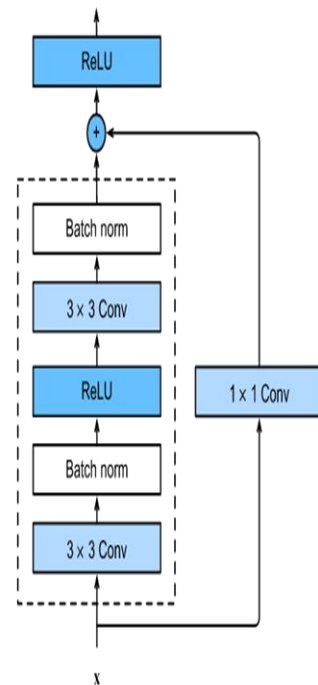


Fig: A Residual Block.

Workings of ResNet.

- **Feature Extraction:**

- **Global Average Pooling (GAP):**

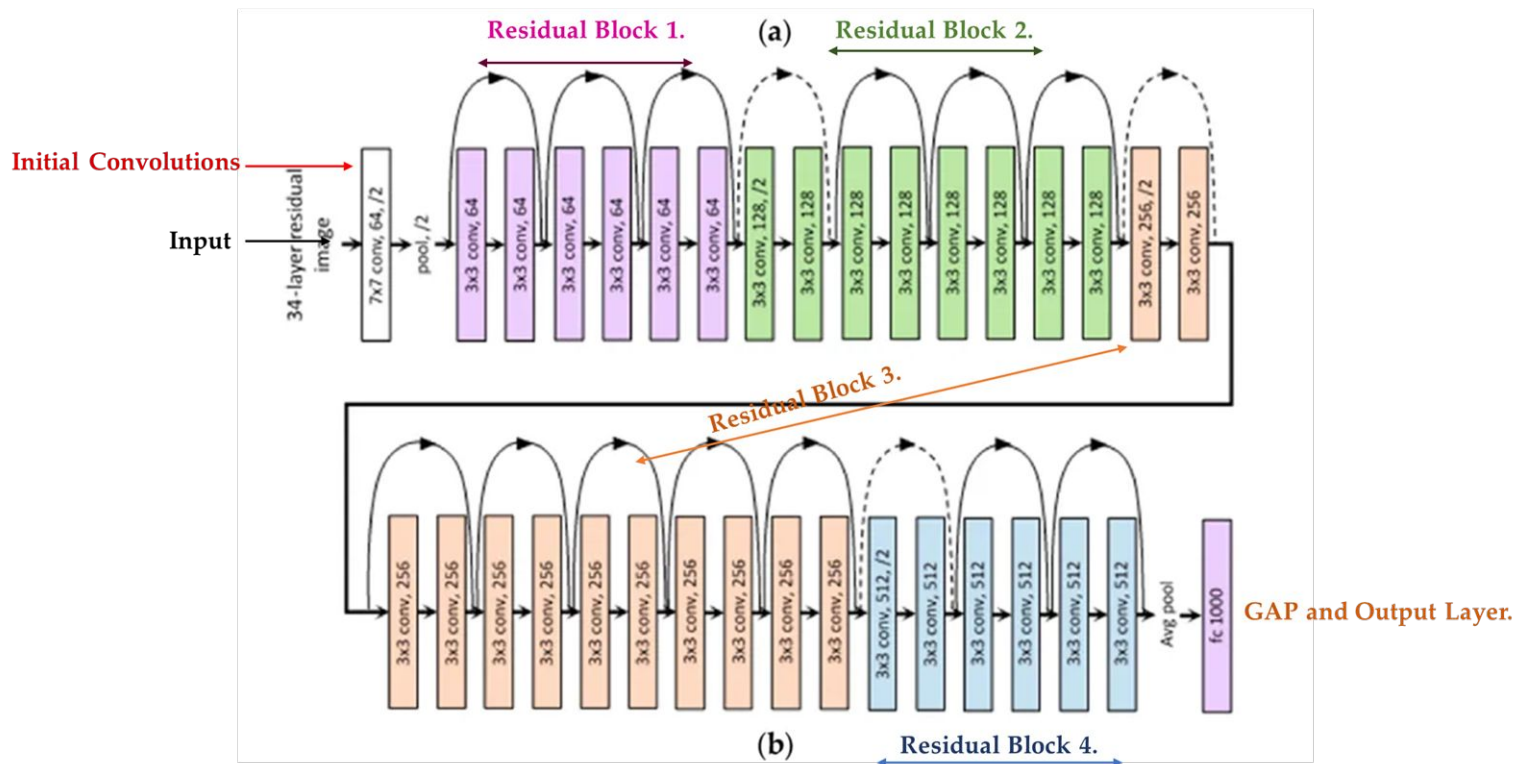
- Instead of using fully connected layers like VGG16, ResNet uses **Global Average Pooling (GAP)** to reduce the feature maps from the convolutional layers.
 - GAP averages each feature map into a single scalar, drastically reducing the number of parameters and preventing overfitting.

- **Classification:**

- **Softmax Layer:**

- The output from GAP is a 1D vector, which is passed into the **softmax layer**.
 - The softmax function converts the feature map into a probability distribution over the possible classes.
 - The class with the highest probability is selected as the predicted label.

ResNet Architecture.



ResNet – 50 Parameters.

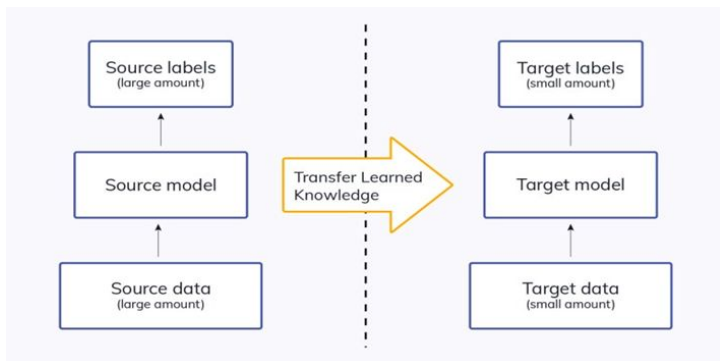
Component	Parameters
Initial Convolutions	9,472
Residual Blocks 1 (Conv2_x)	173,184
Residual Blocks 2 (Conv3_x)	986,112
Residual Blocks 3 (Conv4_x)	5,907,456
Residual Blocks 4 (Conv5_x)	11,814,912
Final Layers	2,049,000
Total	25.0M Parameters

Better Result in Practice.

Transfer Learning

Transfer Learning.

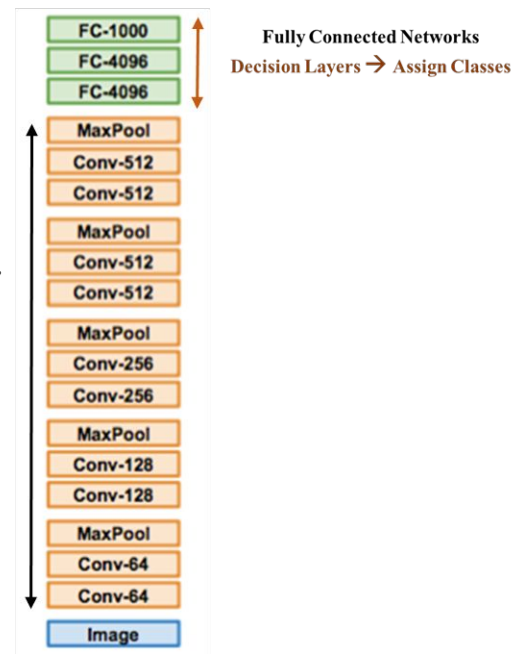
- **Improvement of learning** in a **new task** through the **transfer of knowledge** from a **related task** that has already been learned {**Trained**}.
- Cheaper, faster way of **adapting a neural network by exploiting** their **generalization properties**
- Training your own model can be difficult with limited data and other resources
 - It is a laborious task to manually annotate your own training dataset
 - Why not reuse already pre-trained models?



Transfer Learning-Process.

- Start with pre-trained **base network**.
 - Base model is initiated using popular architecture such as ResNet/VGG etc.
 - Generally Trained on Imagenet Dataset.
 - Downloads the pre-trained weights.

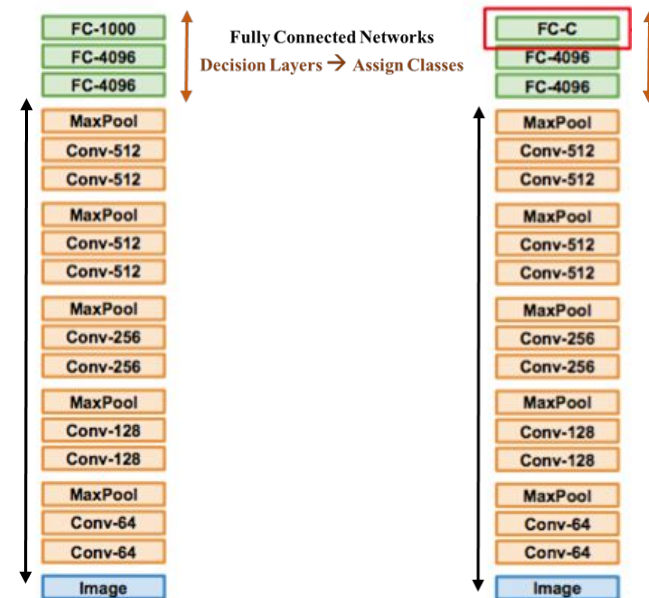
**Convolution Operations For
Feature Extractions.**



Transfer Learning – Process.

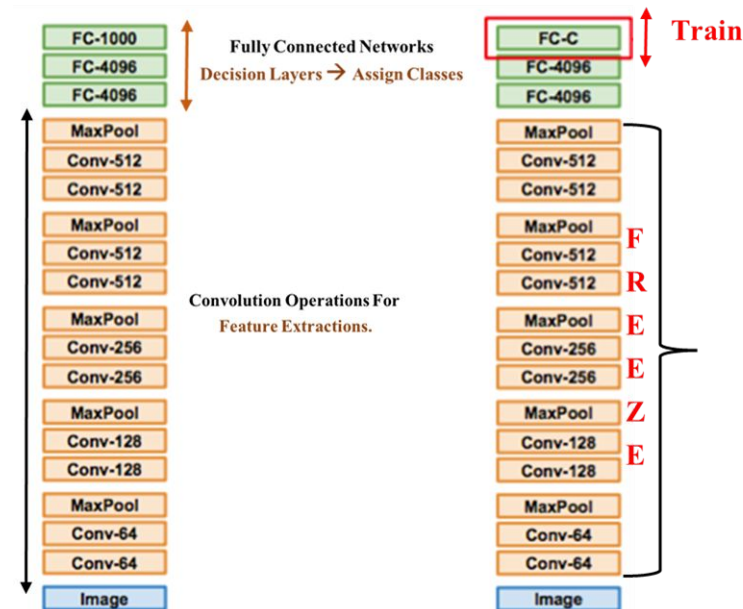
- **Redesign** a decision layer of base model, i.e. add a final layer that is compatible with problem in hand.
 - **New dataset with C classes**

Convolution Operations For
Feature Extractions.



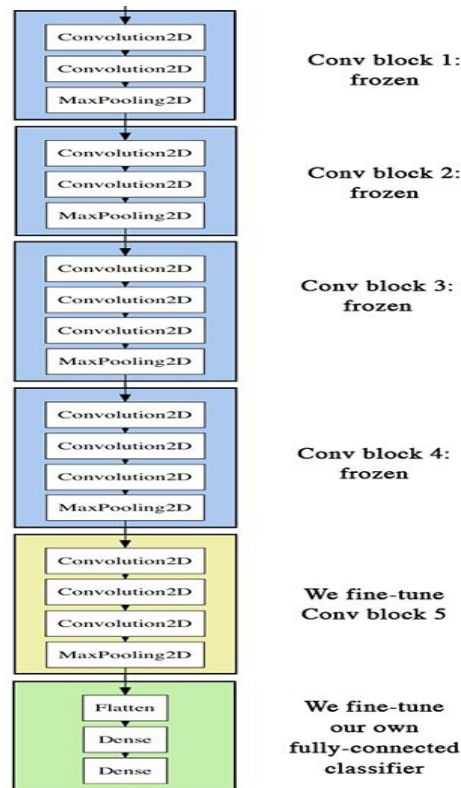
Transfer Learning – Process.

- **Redesign** a decision layer of base model, i.e. add a final layer that is compatible with problem in hand.
 - **New dataset with C classes**
- Training of the Model:
 - Two options:
 - **Options 1: If the dataset is Small:**
 - Freeze Weights and **{Fine Tune}** more layers of base network.
 - Since the dataset is small; no need to train on many layers unfreezing few layers may work.
 - **Use a Small Learning Rate:**
 - This is crucial!
 - A **small learning rate (e.g., 1e-4 or 1e-5)** prevents drastic weight updates and helps retain useful features learned from the original dataset.



Freezing and Fine Tuning.

- Freeze layers do not change weights during training.
- Fine-tuning is done by unfreezing the base model or part of it.
- Re-train the entire model again on the whole dataset at a very low learning rate.
- The low learning rate will increase the performance of the model on the new dataset while preventing overfitting.
- Which layer to re-train?
 - Depends on the domain
 - Start by re-training the last layers (last full-connected and last convolutional)
 - work backwards if performance is not satisfactory



Where to find Pre-trained models?

- Keras pre-trained models:
- Classify with base classes (imagenet class) with ResNet.

```
from tensorflow.keras.applications.resnet50 import ResNet50
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.resnet50 import preprocess_input, decode_predictions
import numpy as np

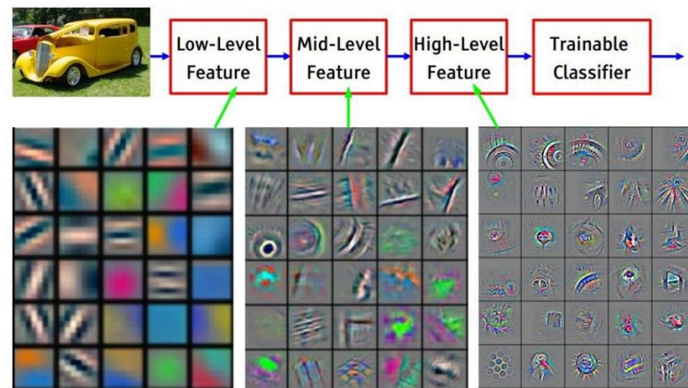
model = ResNet50(weights='imagenet')

img_path = 'elephant.jpg'
img = image.load_img(img_path, target_size=(224, 224))
x = image.img_to_array(img)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x)

preds = model.predict(x)
# decode the results into a list of tuples (class, description, probability)
# (one such list for each sample in the batch)
print('Predicted:', decode_predictions(preds, top=3)[0])
# Predicted: [(u'n02504013', u'Indian_elephant', 0.82658225), (u'n01871265', u'tusker', 0.1122357
```

Why it Works?

- Intuitively, deep networks learn multiple levels of abstraction i.e. Different layer of convolution learns/extract different features
 - Generally low level to high level features
 - Low level features such as lines, spot and edges will remain the same for all kind of images



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

Closing Remarks.

- Setting up a problem:
 - Obtain training data
 - use appropriate representations for inputs and outputs.
- Choose network architecture
 - More neurons need more data
 - Deep is better, but training may be unstable
- Choose appropriate loss function
- Choose regularization and other heuristics like batch norm, dropout etc.
- Choose optimization algorithm.
 - E.g. Adam {Discuss this in Workshop}
- **Read a lot.**
- Happy Training!!!!

Thank You!